https://hjfy.top/arxiv/2610.18867v1

EmbodiedEvo：面向物理智能的自进化编程智能体

Wenyi Li[†], **Yizhe Chi, Youjie Zheng, Chunyu Liu, Xiaoqiu Wang, Mingju Gao, Zheng Jiang, Deyao Hong, Calvin Xiao, Kaisen Yang**[‡], **Qinhuai Na**[‡]

Navers Lab, Einsia.AI | Tsinghua University

[†]Project Lead [‡]Corresponding Authors

将通用智能体引入物理世界需要一种可执行的物理智能表示。与直接将观测结果和语言指令映射到动作的视觉-语言-动作（VLA）模型相比，代码能够显式表达控制逻辑并实现复用。代码不仅可作为策略的表示，还可作为智能体完成任务的工作空间。智能体无需遵循预设的规划、执行与验证工作流，而是能够自主检查状态，修改并执行代码，并利用反馈选择下一步行动。这种基于软件工程（SWE）风格的环在数字环境中取得成功，激发了其向物理任务的扩展。然而，物理执行仍然容易重复失败。为保留并优化后续场景的经验，我们提出 **EmbodiedEvo**：一个编码智能体在单一游乐场场景中通过反馈不断演化出可复用的技能（Skills）与经验（Lessons），随后冻结所得框架以应用于多样新场景。在 RoboDojo-Sim 上，EmbodiedEvo 实现了平均得分 44.48，成功率 37.82%，超越所有官方验证结果。配对测试显示，其在依赖记忆的任务上提升最为显著，且能成功复用协调性操作流程；但精细接触控制仍具挑战。更强模型积累的经验也能提升较弱执行器的表现：Astra 的经验使 Sol 和 Luna 的成功率分别提升了 8.86 和 5.50 个百分点。相较于原始交互历史，使用技能与经验可在任务表现相当的情况下，将执行成本降低 53.0%。

Date: October 13, 2026

Homepage: https://lab.einsia.ai/embodiedevo

Correspondence: liwenyi@einsia.ai, kaisenyang@einsia.ai, nana@einsia.ai

Einsia



Figure 1 具身进化（EmbodiedEvo）概述。在特定任务的游乐场交互过程中，编码智能体通过物理反馈开发控制程序，并修订可复用的技能与经验。由此产生的框架被冻结，并由新场景中的全新测试智能体继承。

1 引言

“智能由与世界的互动动态决定。”

— 罗德尼 · A · 布鲁克斯

智能体正开始超越数字世界，进入现实世界的物理环境。大型语言模型使智能体能够在软件环境中通过推理、使用工具以及基于反馈迭代开发解决方案 [1]。将这种自主性扩展到物理任务，要求智能体能够将推理与感知和执行相结合，同时适应其动作带来的后果。将这些能力应用于机器人领域，需要一种智能体可以执行、检查和优化的物理知识表示。

与视觉-语言-动作（VLA）模型相比，后者将观测结果和语言指令直接映射到机器人动作 [2, 3]，代码使控制逻辑变得明确、可编辑且可复用。代码能够以智能体可以检查和更新的形式表示任务相关的对象、估计状态、关系以及进展。我们将这定义为“代码即世界”。代码还为开发解决方案提供了工作空间。早期的代码即策略系统在预定义的生成与执行框架内合成程序 [4]。软件工程风格（SWE 风格）的智能体通过自主地进行检查、编辑、执行和诊断来扩展这一过程 [1]。应用于机器人领域 [5, 6]，这种自主性使智能体能够决定何时观察、执行哪段代码，以及如何验证结果并从失败中恢复。我们将这种由智能体主导的交互定义为“智能体即策略”。

可靠的物理执行不仅需要可编辑的代码：程序必须基于机器人的观测、动作规范和物理约束。即使是最先进的 GPT-6 Astra 也可能因不准确的接近几何、时机错误的修正以及手臂间协调不佳而失败 [7]。交互过程揭示了有用的动作、容差和检查方法，但若知识仅局限于单次尝试，则在后续场景中必须重新发现。我们的基准迹线（第 3.3节）识别出反复出现的失败，这些失败需要可复用的进度检查、物理后条件和恢复流程。这些观察结果促使我们演化测试框架，使后续智能体能够建立在先前的交互基础之上。基于反馈驱动的技能最优化和测试框架精炼 [8–10]，测试框架演化在保持模型权重和机器人接口不变的同时，保留并修订控制代码与经验。

为此，我们引入了 **EmbodiedEvo**，这是一个自我改进的编码智能体，通过可复用的技能和经验实现演化。我们利用每个任务中的单一场景为该智能体构建了一个训练场（Playground）。在此场景中，智能体在持续的仿真会话中执行代码，并通过物理反馈迭代优化其技能与经验。技能存储控制流程及其使用条件，而经验则保留观察到的失败模式、约束条件以及恢复过程。随后，我们冻结生成的测试框架，并将其提供给独立的测试智能体，这些智能体在相同任务的不同布局中运行，且均与训练场布局不同。每个智能体从全新的工作空间、Python 状态和仿真器开始，使用只读的测试框架开发本地程序，同时持续进行观察、执行和恢复。

RoboDojo [7] 仿真任务的实验表明，继承的经验能够提升后续执行效果。在任务描述和维基食谱的帮助下，Astra/xhigh 在每个任务的 50 个测试场景中达到了 37.82% 的平均成功率，相较于已发表的 Astra 结果（22.48%）有显著提升。配对测试保持执行器和场景固定，仅使用任务描述便将成功率从 8.50% 提升至 30.17%。在依赖记忆的任务上取得最大提升，同时成功复用了如积木堆叠等协调性操作；精细插入和液体处理仍然困难。经验还能在不同模型间迁移：Astra 的技能与经验使 Sol 和 Luna 的成功率分别提升了 8.86 和 5.50 个百分点，相较于无经验的情况。这些结果区分了生成有用控制知识的能力与应用该知识的能力。

我们的贡献是：

- **通过不断演化的束缚装置实现物理自我提升**。我们提出了 EmbodiedEvo，将物理交互转化为持久的控制流程和决策指导。*代码即世界*，*智能体即策略*使这一知识变得明确、可编辑且可重用，同时模型权重和原生机器人接口保持不变。

- **将动作与其条件相耦合的经验**。我们的工具链将可执行的技能与关于适用性、故障模式及结果验证的课程相结合。一个从沙盒到测试的协议在每个任务中获取单一场景的知识并将其冻结，供独立智能体使用，从而可在未见过的布局和不同模型之间复用，而无需额外的沙盒探索。

- **获取经验与应用经验是两种不同的能力**。我们表明，未完成的任务也能产生有用的知识，而执行能力较弱的代理可以从自身难以获取的经验中获益。EmbodiedEvo 在平均得分和成功率上均超越了所有官方验证的 RoboDojo 仿真结果；在测试成本降低 53.0% 的情况下，Skills/Lessons 仍保持与原始历史相当的性能。

2 相关工作

通用机器人控制模型。 VLA 模型将语义知识、具身推理、异构数据和交互经验整合到学成策略中 [2, 3, 11, 12]。RoboDawn 利用上下文演示来确立机器人界面的使用，而早期的 Astra 评估揭示了直接控制中的几何与协调失败 [7, 13]。编码方法使行为可编辑：CaP-X 和 EmbodiedSWE 通过交互开发程序，RAPID 将演示导出的程序适应场景几何结构，闭环软件最优化则利用存档的实现来获取策略 [5, 6, 14, 15]。HexaAnything 将编码的状态与策略同感知和控制工具相结合 [16]。EmbodiedEvo 将这一程序级适应性与可复用的经验相结合，使得智能体在修订新场景代码时可以借鉴先前的交互经验。

自进化智能体与工具链。自我改进涵盖了在 R-Zero 和 SkillRL [17, 18] 中的策略学习，以及在 SkillOpt、MemEvolve 和 SHAPER [8, 19, 20] 中对技能、记忆实现和上下文构建代码的修订。Continual Harness 与 RegenHarness 研究工具链的优化与受控修订 [9, 21]；Hyperagents 修订任务与元智能体程序，而 EnvHarness 则定制周围环境 [22, 23]。CORAL 支持异步探索，并共享笔记与技能 [10]。我们的重点在于工具链中存储的物理控制知识：可执行的过程及其适用时机、失效原因以及恢复方法。

可复用的物理经验。 RATs 在下游任务到来之前获取并冻结代码技能，包括跨编码智能体的迁移 [24]。RoboSkill 将可复用的代码与文本指导结合触觉反馈，而 ASPIRE 在任务序列间演化技能库 [25, 26]。RoboHarn-Evo 更新分层物理知识，EmbodiedRSI 选择实验以区分代码与技能假设，PhysicalRSI 通过具身评估选择适配的后续步骤 [27–29]。我们将经验获取与执行分离，测试来自某一场景的知识是否能帮助同一模型或不同模型解决相同任务的新布局。

3 具身进化

3.1 代码即世界

Code as World 将与物理环境的交互组织为两个层次的代码。L1 提供观察和动作的基本操作；L2 将这些操作组合成可执行的 Python 过程。图 2 将这种程序化控制置于更广泛的具身控制范式之中。

L1：环境原语。 环境暴露了四个基本操作，用于读取任务、观察场景、执行动作和重置环境（表 1）。这些操作构成了智能体可用的固定接口。重置操作仅限于游乐场（Playground）中使用，它允许在相同场

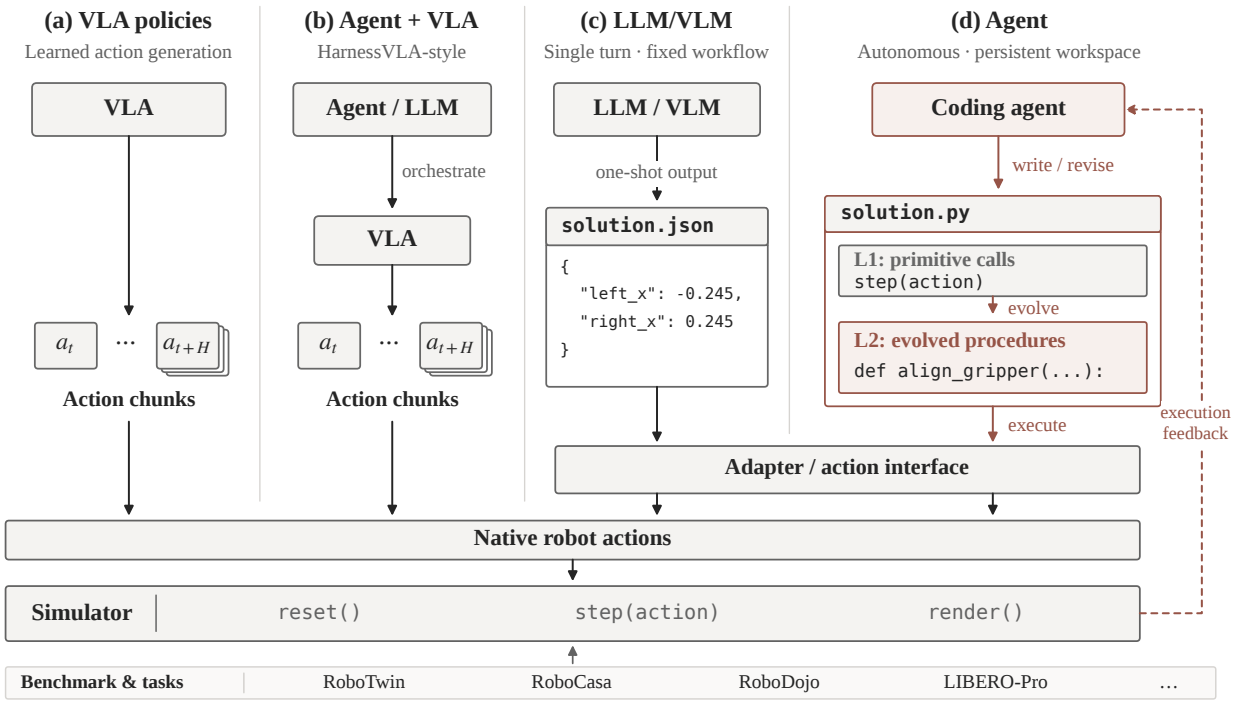


Figure 2 具身控制范式的比较。(a) VLA 策略生成原生动作块。(b) 智能体协调 VLA 策略，同时保留其动作生成角色。(c) 单轮 LLM/VLM 生成结构化的动作规范供适配器使用。(d) 编码智能体通过环境反馈开发并修订可执行程序，将基本调用（L1）组合为演化的过程（L2）。虚线环表示程序修订的执行反馈。

景下进行另一次尝试。

Table 1 原生环境接口。前三个原始操作在两个阶段均可用；reset 仅限于游乐场阶段使用。

Primitive	Function
`get_instruction()`	Returns the current task and goal.
`get_observation()`	Returns camera images and robot state without advancing physics.
`step(action)`	Executes a native action and returns the resulting observation, completion signal, and termination status.
`reset()`	Restores the same scene for another exploration attempt and renews its native action allowance.

在 RoboDojo 中，观测包含 RGB 图像和机器人状态，包括关节状态、末端执行器位姿以及夹爪命令状态。动作指定关节或末端执行器的目标连同夹爪命令；原生控制器负责处理逆运动学和插值。通过此接口无法获取物体的真实位姿、深度以及相机校准信息。

L2：由基本元素组成的程序。 智能体编写结合 L1 调用的 Python 程序，通过变量、函数、环和条件分支实现。例如，抓取过程可以读取当前机器人状态，朝目标移动，夹紧夹爪，然后抬起。该程序可接受目标坐标作为参数，并在继续之前检查观测到的末端执行器位姿。因此，一个程序可以协调多个原生动作，其函数可在后续程序中重复使用。

这种分离方式确定了机器人可观察和执行的内容，同时让智能体决定如何组合这些能力。代码显式地定义了执行顺序、参数和检查步骤，使智能体能够在任务进行过程中检查并修改执行流程。图 3 展示了这

些程序如何与任务环境连接。程序层面的入口点 `step(code)` 通过适配器执行提交的 Python 代码，程序会向选定的模拟器发出原生的 `step(action)` 调用。适配器确保了在不同任务后端间接口的一致性；我们的实验使用 RoboDojo。

3.2 智能体即策略

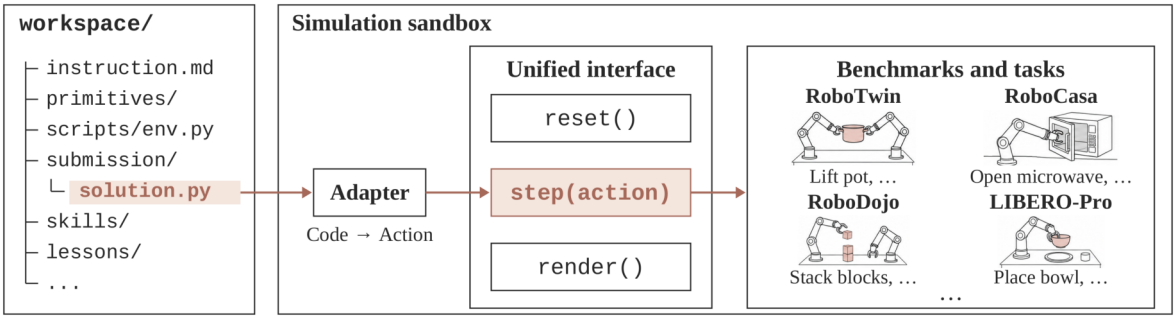


Figure 3 通过统一仿真接口实现代码到任务的映射。智能体在其工作空间内开发 `submission/solution.py` 并提交给适配器。适配器执行该程序，并通过统一接口向选定的基准后端发出原生的 `step(action)` 调用。

智能体作为策略，将控制决策分配给编码智能体：观察什么内容、执行哪个程序，以及在收到反馈后如何修改程序。通过 L1/L2 接口，智能体引导一个迭代过程，包括检查、编程、执行和修复（图 2d）。它根据当前的观测结果和执行历史，选择任务分解方式以及每一步的执行时机。

一个通过代码进行操作的工作空间。 智能体在隔离的文件系统工作区中运行（表 2）。任务说明和接口文档提供了固定规范；源文件和临时空间支持程序开发。执行记录保存了运行过的操作及发生的情况，使智能体能够检查之前的尝试。技能和经验提供了第 3.4 节引入的持久化知识。

Table 2 软件工程工作区。路径相对于智能体的工作区根目录。协议文件和执行记录对智能体是只读的。

Component	Role and access
`instruction.md`	Fixed task delegation, phase rules, and budgets.
`primitives/primitives.md`	Fixed documentation of the robot interface.
`submission/solution.py`	Editable Python source for the next control segment.
`scripts/env.py`	Fixed client for execution, status queries, and session completion.
`observations/`	Host-written history of submitted source, observations, video, and execution logs.
`scratch/`	Writable plans, calculations, and temporary analyses.
`skills/`, `lessons/`	Reusable procedures and diagnostic knowledge; writable in Playground and read-only in Test.

智能体主导的交互环。 智能体读取任务并检查场景，编辑 `submission/solution.py`，并通过执行客户端提交（图 3）。执行后，它可以检查提交的源代码、相机观测结果、标准输出和错误信息、完成信号以及剩余预算。随后，它决定是否获取新的观测、调整目标、修改函数、进入下一阶段，或结束回合。例如，在观察到物体掉落之后，它可以检查新场景，并在再次尝试抓取前修正抓取策略。这些决策均在一次编码-智能体会话中完成。

适应当前的生理状态。 机器人状态、Python 变量和函数定义在提交之间持续存在。编辑代码或遇到错误不会撤销已完成的动作，因此智能体必须根据当前场景调整其下一个程序。Playground 允许在同一场景中重置，并重新获得原生动作额度，而执行槽位和经过的时间则保持累积。Test 禁用重置，要求在当前场景和剩余预算内完成恢复。因此，工作区支持在整个回合中进行本地程序修订；第 3.4 节解释了如何在回合之间保留经验。

3.3 深入剖析阿斯特拉的失败

为了理解编码智能体在物理交互中的薄弱环节，我们评估了在代码工作区中采用高推理努力的 GPT-6-Astra 智能体，该工作区如第 3.2 节所述。评估任务涵盖 RoboDojo-sim 中的泛化、准确率、长时程、记忆和开放性任务，每类任务包含 5 个场景。Astra 接收任务描述和机器人接口文档，但不包含 RoboProbe [30] 中描述的任务维基。每个回合从一个全新的智能体、工作区和模拟器开始，允许在任务的原生动作限制内最多执行 100 次被接受的代码，且禁用重置。

Astra 的平均成功率达到了 8.50%。我们检查其提交的程序、摄像头观测结果以及反馈，以定位在物理执行过程中合理计划失败的位置。图 4 展示了两个具有代表性的回合，每个回合揭示了发出有效指令与任务推进之间存在的不同差距。

正确的动作序列仍然缺少可靠的操纵流程。 在井字棋游戏中，Astra 尝试拿起一个环并将其放置在棋盘上。夹爪到达了目标位置，但连续两次尝试后环仍留在桌面上（动作 216 和 283）。在检查腕部摄像头画面并调整抓取姿势后，Astra 在动作 367 首次明显成功持住环，消耗了其 1,100 个动作预算的三分之一。随后回合因超时而未完成棋盘布置。

此迹线表明，构建局部操作流程可能与任务完成产生竞争。这促使保留一个参数化的操作程序，该程序结合接近、对齐、闭合及物体抓取验证步骤，使后续尝试能够基于已成功运行的操作进行优化。

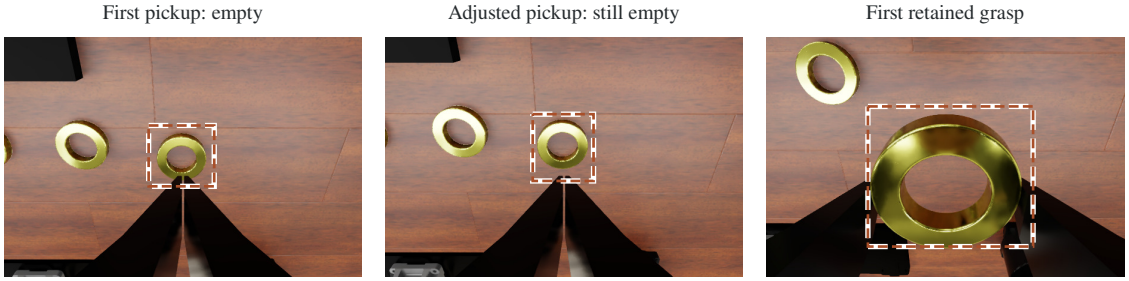
动作决策缺乏显式的前置条件检查。 在模仿排序中，Astra 在动作 240 时仅识别出前三个物体，却已经开始接近第一个物体，而示范仍在进行。回合在 16 个动作后终止，导致 1,344 个动作未被使用。可见的差距在于：未检查示范是否已完成便采取动作。仅识别目标并不能说明已准备好对其进行操作。这促使我们保留基于观察的指导，明确在行动前需验证的内容，未解决的解释则标记为假设。在此情况下，迹支持检查示范是否完成；它并不证明早期移动导致了终止，也不保证等待就能确保成功。

两次失败暴露了互补的需求：(1) 执行操作的可靠流程，以及 (2) 判断何时尝试该操作的明确条件。在回合内，物理反馈可以揭示这两点，但每个新的 Astra 智能体都需从接口和任务描述重新开始。这促使我们设计一种机制，将局部修正转化为可复用的流程，并保留失败条件以指导后续决策。第 3.4 节通过演化进化来发展该机制。

3.4 通过驾驭演化实现自我提升

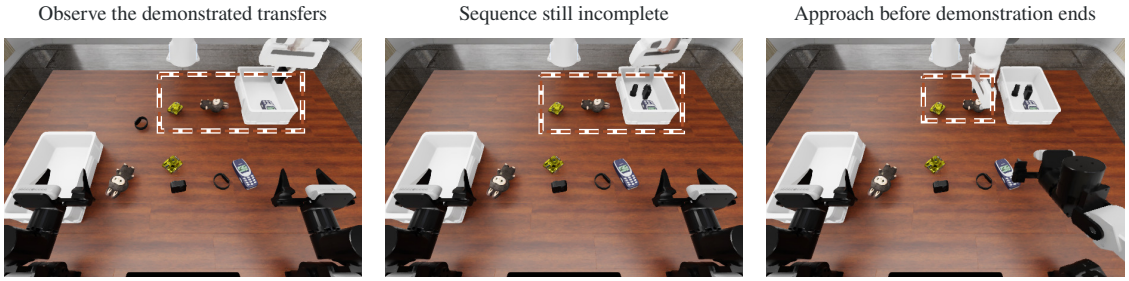
EmbodiedEvo 通过保留不断演化的工具箱中的 *技能* 和 *经验教训* 来满足第 3.3 节中的两项需求（图 5）。技能将固定的原生基本操作组合成可重复使用的控制流程，例如带有检查的提升抓取。经验教训记录了适用条件、观察到的故障模式以及诊断和恢复的指导建议。它们共同捕捉了如何执行一项操作，以及在操作前后应检查的内容。自我改进会更新这些知识，而模型参数、任务目标和机器人接口则保持不变。

(a) Tic-tac-toe: repeated empty lifts consume the action budget



The arm reaches its targets, but useful contact must still be established.

(b) Imitation sorting: acting before the demonstration ends



Knowing the next object does not establish that manipulation can start.

Task	Score	SR (%)
Tic-tac-toe	55	0
Imitation sorting	10	0

Figure 4 Astra 的两种失败轨迹。 (a) 在首次保留抓取之前有两个空的提升动作；随后回合耗尽预算但未完成。(b) 观察在未完成的演示过程中切换到操作，随后终止。方框标识相关物体。表格报告了 Astra 在每项任务所有测试场景中的平均表现。

在图 5 的左侧，一个 Playground 场景中持续进行代码编辑、执行和反馈的环。智能体状态和 Python 定义在提交之间保持不变；重置允许在同一场景中再次尝试。修订与复用箭头展示了智能体如何更新技能和经验，并在后续尝试中应用它们。一次成功的操作可以生成一个参数化的程序；一次失败的尝试可能揭示有用条件或提示恢复策略。即使任务未完成，也可能得出有价值的发现；暂时的解释仍为假设。

冻结箭头标记了沙盒阶段的结束：只有技能和课程被作为只读资源带入测试阶段。在右侧，同一任务的每个未见过的场景都会从一个全新的智能体、工作区、Python 命名空间和模拟器开始。智能体使用冻结的框架来构建代码，并通过执行反馈来调整代码。重置功能被禁用，测试回合不会更新框架或共享新的经验。这将场景内的适应过程与沙盒阶段所获取知识的贡献分离开来。

相同的冻结成果可被另一个执行器使用，因为其内容是明确的程序和指导。跨模型复用无需额外的 Playground 探索、参数更新或学成适配器。我们在相同的执行器和测试条件下，比较自生成与转移的 harness，以评估所获取的知识是否在不同模型间依然有用。

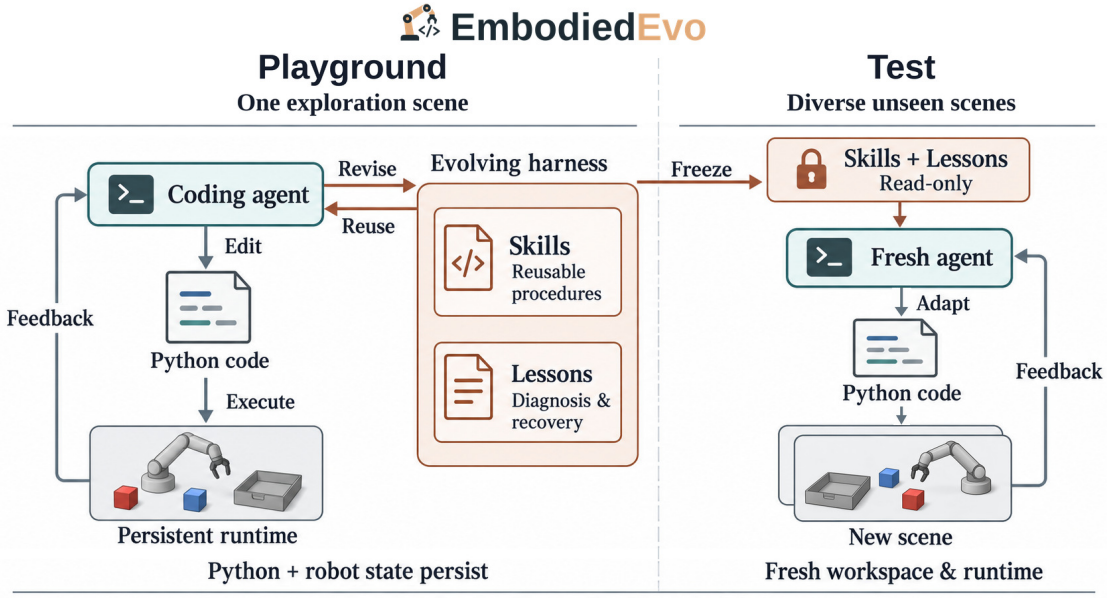


Figure 5 EmbodiedEvo 框架。左图：在一个 Playground 场景中，智能体编辑并执行代码，利用反馈来修订技能和课程，并在后续尝试中复用这些成果。冻结箭头将这些成果转移到测试环节。右图：每个未见过的场景均从一个全新的智能体和运行时环境开始；代码通过反馈进行自适应调整，而测试框架保持只读状态。机器人场景为示意图。

4 实验

我们在 RoboDojo-Sim 上对 EmbodiedEvo 进行评估，以回答以下研究问题：**(1)** EmbodiedEvo 与基于 VLA 和 WAM 的方法相比表现如何？**(2)** 哪些能力通过躯体演化得到提升，哪些任务仍然困难？**(3)** 获得的经验能否在模型和任务之间迁移？

4.1 实验设置

任务与智能体。 RoboDojo-Sim [31] 是一个用于评估通用机器人操作策略的基准。它包含 12 个泛化任务、8 个准确率任务、8 个长时序任务、6 个记忆任务和 8 个开放任务。每个训练流程（技能 + 课程）均在与测试布局不同的 Playground 场景布局上获取。执行器为 GPT-6-Astra（高努力度和中等努力度）、GPT-5.6-sol（高努力度）以及 GPT-5.6-luna（高努力度），分别简称为 Astra、Sol 和 Luna。

评估量表。 我们使用两种评估尺度：(1) 开发集：每项任务 5 个场景（每种条件 210 回合），用于输入比较、消融实验和跨模型分析；以及 (2) 完整集：每项任务 50 个场景（2,100 回合，一个种子），用于完整基准的全面评估。成对比较使用相同的场景。对于每个泛化任务，这两种尺度分别使用 3 个标准场景和 2 个随机场景，以及 25 个标准场景和 25 个随机场景。场景身份遵循公开评估记录 [32]。跨任务迁移使用固定的 12 个目标任务子集，并与完整基准平均值分开报告。

输入和预算。 任务描述用一句话说明目标；官方维基提供了额外的任务规则和操作指南 [33]。完整评估使用 *任务描述 + 维基*来对齐任务信息与官方 GPT-6-Astra 评估标准 [7]。输入消融实验比较两种情景；所有其他配对分析仅使用 *纯任务描述*。每个 Playground 链和测试回合最多允许 100 次被接受的代码执行，并保留任务的原始动作预算。Playground 允许在同一情景下重置；获取的技能和经验随后被冻结。测试在每个场景中启动一个全新的智能体、工作区和模拟器，且经验为只读状态，不支持重置。

Table 3 **完整 RoboDojo 仿真评估**。 EmbodiedEvo 与 2026 年 10 月 9 日排行榜快照中选定的系统一起，按平均得分升序排列。得分和成功率（SR，%）越高越好；加粗标记每列指标中的最佳值。

Model	Type	Average		Generalization		Precision		Long-horizon		Memory		Open	
		Score	SR (%)	Score	SR (%)	Score	SR (%)	Score	SR (%)	Score	SR (%)	Score	SR (%)
GPT-6-Astra	Agent	28.97	22.48	33.36	**30.50**	12.65	4.00	21.45	8.25	43.04	38.67	34.36	31.00
InternW0-Δ	WAM	30.77	23.91	30.09	22.78	31.98	23.25	46.29	29.33	34.67	34.00	10.84	10.17
Simate-beta	VLA	33.95	27.96	35.09	27.95	34.35	26.92	**57.84**	**43.42**	33.33	33.00	9.12	8.50
Awomo-0.5	WAM	35.34	29.64	30.31	23.84	39.65	33.50	40.36	26.75	42.27	41.22	24.09	22.92
PhysicalRSI	Agent + VLA	36.27	31.38	21.30	15.62	38.05	32.50	46.37	37.33	46.74	46.56	28.88	24.92
VPP2	WAM	39.26	32.26	**36.38**	29.50	**43.03**	**35.17**	56.28	37.00	56.01	55.56	4.61	4.08
EmbodiedEvo (Ours)	Agent	**44.48**	**37.82**	34.12	29.67	23.65	13.25	37.20	23.50	**71.90**	**69.67**	**55.52**	**53.00**

指标 我们报告得分（0–100）和成功率（SR，%）。得分根据基准的任务特定评分规则量化任务完成情况，包括部分进展。SR 是满足任务成功准则的回合所占的百分比。

4.2 总体性能

我们使用 Astra/xhigh 在完整的 RoboDojo-Sim 数据集上评估 EmbodiedEvo。我们将类别平衡得分和成功率与 2026 年 10 月 9 日的 leaderboard 快照中的 VLA、WAM 以及基于智能体的结果进行比较。

表 3 显示，EmbodiedEvo 在所列方法中取得了最高的平均得分（44.48）和成功率（37.82%）。相较于所列最强的 WAM 方法 VPP2，其得分高出 5.22 分，成功率高出 5.56 个百分点。与 Simate-beta（VLA）相比，得分和成功率分别领先 10.53 和 9.86 个百分点；同时，相较于 PhysicalRSI（智能体 + VLA），其得分和成功率分别高出 8.21 和 6.44 个百分点。

具身进化在记忆和开放任务上大幅领先。具身进化在记忆任务上取得 71.90 得分 / 69.67% 成功率，在开放任务上取得 55.52 / 53.00%。其成功率在每个类别中均超过列出的次优方法：记忆任务上为 VPP2 的 55.56%，开放任务上为 GPT-6-Astra 的 31.00%。第 4.3节中的案例表明，技能与经验为何适用于这些任务。技能提供了可复用的抓取、携带和放置程序，能够围绕当前指令进行组合；经验则指导观察、序列追踪以及在进入下一阶段前的检查。二者共同将记住当前场景需求与执行操作的流程相连接，支持依赖记忆的序列和基于指令的操控。

智能体在精确操作和长时程任务执行方面仍存在困难。其精确率成功率（Precision SR）为 13.25%，低于 VPP2 的 35.17%，而长时程成功率（Long-Horizon SR）为 23.50%，低于 Simate-beta 的 43.42%。这些任务需要在连续操作中实现精准接触和可靠的执行。第 4.3 节中的失败案例表明，达到目标位姿可能导致抓取不稳固，完成某些阶段后整体任务仍可能未完成。因此，保留的操作流程与指导仍然依赖于可靠的物理执行：接触精度以及在整个操作序列中维持成功操作仍是持续存在的瓶颈。

Table 4 **任务知识与继承经验**。所有行均在匹配场景上使用 Astra/xhigh。每个单元报告得分 / SR (%)。

Method	Input	Average		Generalization		Memory		Precision		Long-horizon		Open	
		Score	SR (%)	Score	SR (%)	Score	SR (%)	Score	SR (%)	Score	SR (%)	Score	SR (%)
Astra	Task description	14.77	8.50	3.19	0.00	12.67	10.00	11.38	5.00	18.12	0.00	28.50	27.50
EmbodiedEvo (Astra)	Task description	**38.64**	**30.17**	22.36	16.67	59.33	56.67	29.88	15.00	29.50	12.50	52.12	50.00
Astra	Task description + wiki	20.67	13.36	12.33	7.64	39.67	36.67	15.00	7.50	19.38	0.00	17.00	15.00
EmbodiedEvo (Astra)	Task description + wiki	**42.42**	**35.08**	36.15	31.25	70.33	66.67	23.12	12.50	33.75	20.00	48.75	45.00

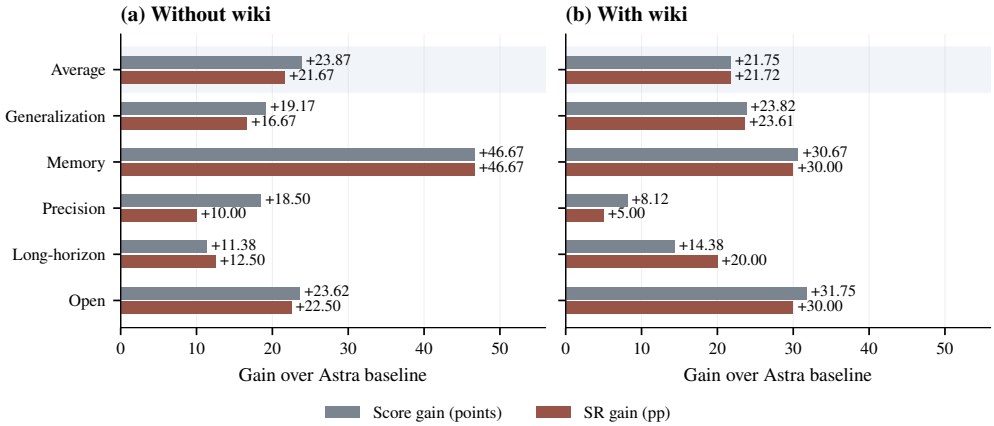


Figure 6 具身进化方法在有无维基百科信息情况下的效果。与 Astra/xhigh 相比，在匹配场景和输入下的得分（分）和成功率（百分点）提升：(a) 仅任务描述；(b) 任务描述 + 维基百科。

4.3 harness 演化究竟改变了什么？

Research questions

(1) 编码智能体在没有演化的情况下已经能够解决什么问题? **(2)** 演化带来了哪些新的能力? **(3)** 演化之后仍然未解决的问题是什么?

为回答上述问题，我们对 Astra/xhigh 在有演化经验与无演化经验的情况下进行了配对评估，使用相同的场景和任务输入。我们测试了两种 wiki 情景 [33]，随后分析任务结果和执行迹，以区分现有的能力、新获得的能力以及仍存在的失败。由于大规模智能体评估的成本较高，我们在 *dev set*（每项任务 5 个场景）上进行此项及所有后续实验。

EmbodiedEvo 在不同输入设置下持续提升性能。在表 4 中，不使用 wiki 时，EmbodiedEvo 将平均得分提升了 23.87 分，成功率（SR）提升了 21.67 个百分点。当两者均提供 wiki 时，得分提升仍为 21.75 分，成功率（SR）提升为 21.72 个百分点。图 6 显示，在两种输入条件下，所有五个类别均获得正向的得分和成功率提升。这些提升表明，无论在何种输入设置下，经验都能提供执行知识；后续的迹线展示了技能如何提供测试操作，以及课程如何指导其使用。

接下来，我们考察这些提升背后的操作和决策。表 5 将代表性任务归类为现有能力、新获得的能力以及尚存的局限性。附录 B 提供了完整的开发集和全集结果。

可见的目标和局部可修正的动作已经可以在无需演化的前提下解决。Astra 在块对齐和传送带匹配任务上分别取得了 60% 的成功率；演化将其分别提升至 80% 和 100%。这两个任务都提供了可见的目标和

Table 5 harness **演化前后任务表现**。选择相同 Astra/xhigh 执行器且匹配开发场景的任务。加粗标记每对“演化前/演化后”中数值较高的项；Δ SR 为百分点变化量。分组描述了这些场景中的观察到的成功情况。

Task	Category	Astra		EmbodiedEvo (Astra)		ΔSR (pp)
		Score	SR (%)	Score	SR (%)	
(1) Successful before evolution						
Align blocks	Open	60	60	80	80	+20
Conveyor matching	Memory	60	60	100	100	+40
(2) Successful only after evolution						
Stack blocks	Generalization	10	0	100	100	+100
Tic-tac-toe	Long-Horizon	55	0	100	100	+100
Imitate sorting	Memory	10	0	90	80	+80
(3) Unsuccessful in both conditions						
Insert tubes	Precision	4	0	36	0	0
Insert key	Precision	9	0	15	0	0

用于调整下一步动作的反馈，使得在线编码和观察能够支持局部修正。在此情况下，演化提升了 Astra 已能执行操作的可靠性。

演化提供了可重用的操作和条件，以推进任务的进展。井字棋和积木堆叠从零提升至 100% 成功率；模仿排序从零提升至 80%。这些任务不仅需要识别空间目标：成功的局部操作必须被重复执行，或者动作必须等待必要的场景信息收集完成后再进行。

图 7 对比了两个匹配场景中的决策点。在井字棋中，基准方法反复抬升却未保持环状物，最终得分为 50 且未能成功。具身进化智能体执行了继承的 `rim_pick` 程序，在动作 38 时保持了环状物，并以得分 100 完成棋盘。这一有效变化是一个可执行的抓取程序，支持重复放置。在模仿排序任务中，基准方法在示范尚未完成时就开始移动，最终得分为 0。而演化出的智能体遵循其观察所得的经验，通过动作 455 记录下当前序列，并成功复现。其顺序与游乐场（Playground）不同，因此可复用的知识关注的是*何时行动*，而非记忆物体序列。这些案例共同将技能（Skills）与可重复的物理操作联系起来，将经验（Lessons）与使用它们的条件联系起来。它们展示了所观察到的机制；成对运行并未隔离单一人工制品的因果效应。附录 D.1 提供了支持性记录，附录 F 给出了完整的语言堆叠迹（Trace）。

可重复使用的行为并不保证精确接触。插入试管的得分从 4 提升至 36，插入钥匙的得分从 9 提升至 15，但两者均未在开发集上成功。图 8 定位了钥匙失败的原因：受限运动在拾取时保持了钥匙的位置，但在交接过程中接收手关闭时为空。后续修正未能将其固定；释放施送手会导致钥匙在插入前就掉落至动作极限位置。因此，一个成功的拾取过程仍使下一个接触转移问题悬而未决。这一成功子程序与完整操作之间的差距解释了为何部分得分提升不必然转化为精度任务上的成功率提升。

4.4 跨模型和任务的演化经验迁移

Research questions

(1) 由更强的模型获得的经验能否提升较弱执行器的表现？**(2)** 来自其他任务的经验能否提升当前任务上的表现？

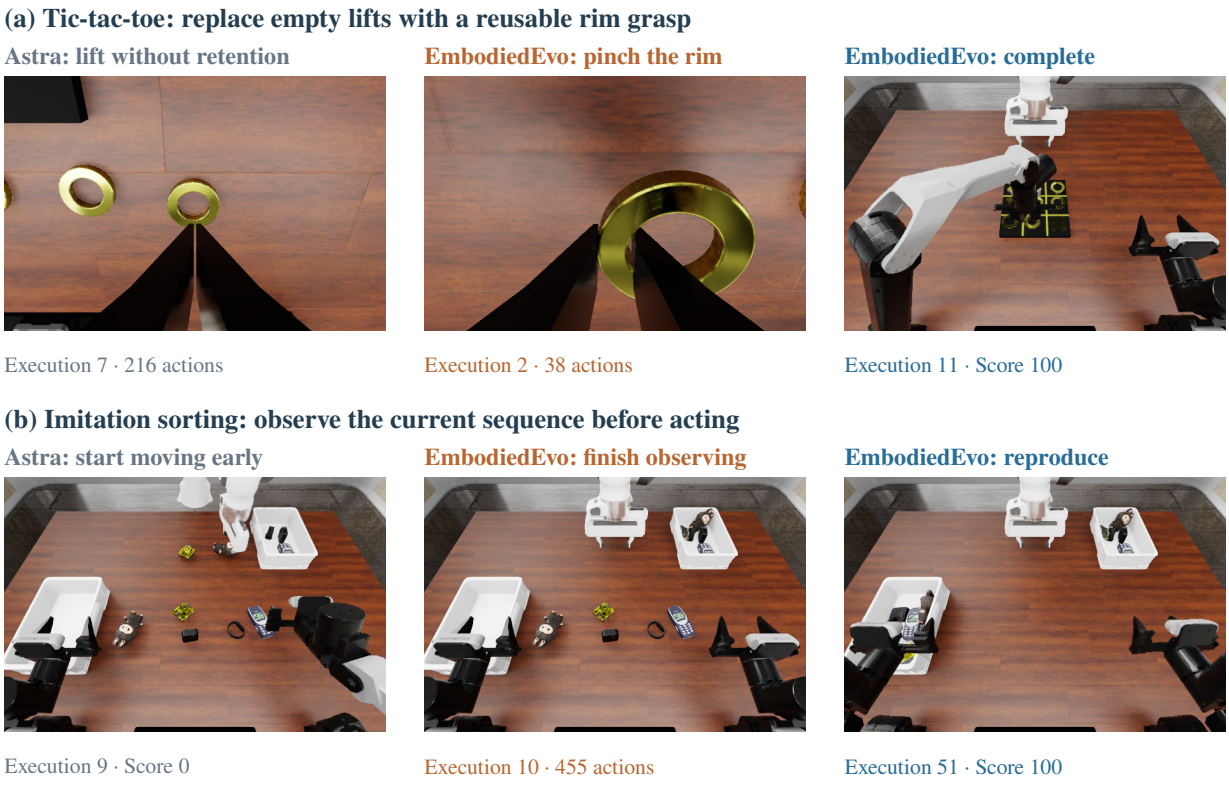


Figure 7　新能力：在决定性操作或阶段的变化。每行对比 Astra 与 EmbodiedEvo 在相同任务和场景下的表现。(a) 井字棋，布局 27：空抓取与继承的边缘抓取及完成的棋盘。首次抓取使用了不同的环和机械臂。(b) 仿照排序，布局 4：早期动作与观察完整新顺序后再执行。两者在时机和动作上均不同；基准反馈无法识别终止原因。帧显示选定的决策点，而非同步时间。



Figure 8　剩余限制：未能成功传递接触。 EmbodiedEvo，插入钥匙，布局 44。捐赠者保留钥匙，但接收者关闭时未抓住杆身（高亮）；最终释放导致钥匙丢失（高亮）。手腕观测和夹持器反馈表明失败发生在交接阶段，插入之前。此回合得分为 15，无成功，与任务开发集的平均分一致。

为回答上述问题，我们针对每个接收执行器，进行了无经验、自演化束带与 Astra/xhigh 束带的匹配比较。同时，我们将 Astra/xhigh 固定在十二个目标任务上，对比相同任务、相关任务和不匹配任务的束带与无经验情况。所有迁移的技能与经验均被冻结，接收方不再进行进一步探索。

更强的经验获取对较弱的执行器更有益。表 6 和图 9 在固定执行器的情况下，对比了无经验（无演化）、自演化测试套件以及来自 Astra/xhigh 的测试套件。相较于无经验，Astra/xhigh 的经验使 Astra/medium

Table 6 **三种经验条件下的性能表现**。得分和成功率（SR，%）越高越好。每个执行器在不同条件下使用匹配的场景；所列执行器获取自演化的工作台；最终组复用来自 Astra/xhigh 的冻结经验。

Executor	Average		Generalization		Precision		Long-horizon		Memory		Open	
	Score	SR (%)	Score	SR (%)	Score	SR (%)	Score	SR (%)	Score	SR (%)	Score	SR (%)
No Experience (No Evolution)												
Astra / xhigh	14.77	8.50	3.19	0.00	11.38	5.00	18.12	0.00	12.67	10.00	28.50	27.50
Astra / medium	6.58	4.33	1.81	0.00	2.88	0.00	5.88	0.00	7.33	6.67	15.00	15.00
Sol / xhigh	4.10	3.17	0.56	0.00	3.00	2.50	3.12	0.00	3.33	3.33	10.50	10.00
Luna / xhigh	0.08	0.00	0.42	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
Self-Evolved Harness												
Astra / xhigh	**38.64**	**30.17**	**22.36**	**16.67**	**29.88**	**15.00**	**29.50**	**12.50**	**59.33**	**56.67**	**52.12**	**50.00**
Astra / medium	16.05	8.67	6.49	0.00	6.38	0.00	16.50	0.00	28.00	23.33	22.88	20.00
Sol / xhigh	4.09	2.50	0.28	0.00	0.38	0.00	12.12	5.00	0.17	0.00	7.50	7.50
Luna / xhigh	0.62	0.00	0.00	0.00	0.00	0.00	3.12	0.00	0.00	0.00	0.00	0.00
Harness from Astra/xhigh												
Astra / medium	30.60	23.06	17.67	15.28	11.38	0.00	26.12	12.50	55.83	50.00	42.00	37.50
Sol / xhigh	14.09	12.03	4.51	3.47	2.75	0.00	15.38	10.00	27.33	26.67	20.50	20.00
Luna / xhigh	6.55	5.50	0.21	0.00	0.38	0.00	4.50	0.00	20.17	20.00	7.50	7.50

备注：执行者标签包含推理努力。泛化变体、每个类别内的任务以及五个类别在其相应层级上具有相等的权重。场景选择和评估细节总结见附录 B。

的成功率（SR）提升了 18.72 个百分点，使 Sol/xhigh 提升了 8.86 个百分点，使 Luna/xhigh 提升了 5.50 个百分点。Sol 的成功率从其自身测试套件的 2.50% 提升至 12.03%；Luna 则从零提升至 5.50%。Astra/medium 测试的是源推理努力的变化，而 Sol 与 Luna 测试的是源模型的变化。这些提升将获取有用经验的能力与应用经验的能力分离开来：较弱的执行器可以使用其自身探索未能建立的程序和条件。

转移的经验为控制提供了可操作的条件。在 `cover_blocks` 任务中，Sol 在使用自身约束条件时，在 680 次原生动作后失败，但使用 Astra/xhigh 的约束条件后仅用 678 次动作就成功了（图 13b）。其自身程序在横向移动过程中丢失了杯子；而转移后的约束条件能够区分空杯与覆盖方块的杯子的抓取高度。Sol 利用这些条件，并根据当前场景调整颜色顺序。该案例将成功的复用与选择适当的动作及其在序列中的位置联系起来，且动作消耗几乎不变。

其他任务的经验在无需目标任务探索的情况下提升了性能。在十二个固定目标上，相关任务的经验相较于无经验情形分别带来了 14.18 分的得分提升和 14.72 个百分点的成功率提升；不匹配任务的经验则分别带来 18.65 分和 16.67 个百分点的提升（图 10）。相同任务的经验提供了 24.43 分的得分参考提升和 21.39 个百分点的成功率参考提升。两种迁移情景均复用了现有资源而未对目标情景进行探索，其提升效果与通用控制知识的复用一致，包括有限运动、状态保留检查以及终端返航检查等，这些知识可指导不同任务间的执行。

仅任务标签相似度无法预测迁移收益。在此目标集上，不匹配的迁移比相关迁移高出 4.47 得分点和 1.94 SR 点。不同任务的方向各异：在 `swap_T` 上，不匹配和相关源分别产生 Δ SR 的 +100 和 0 分；在

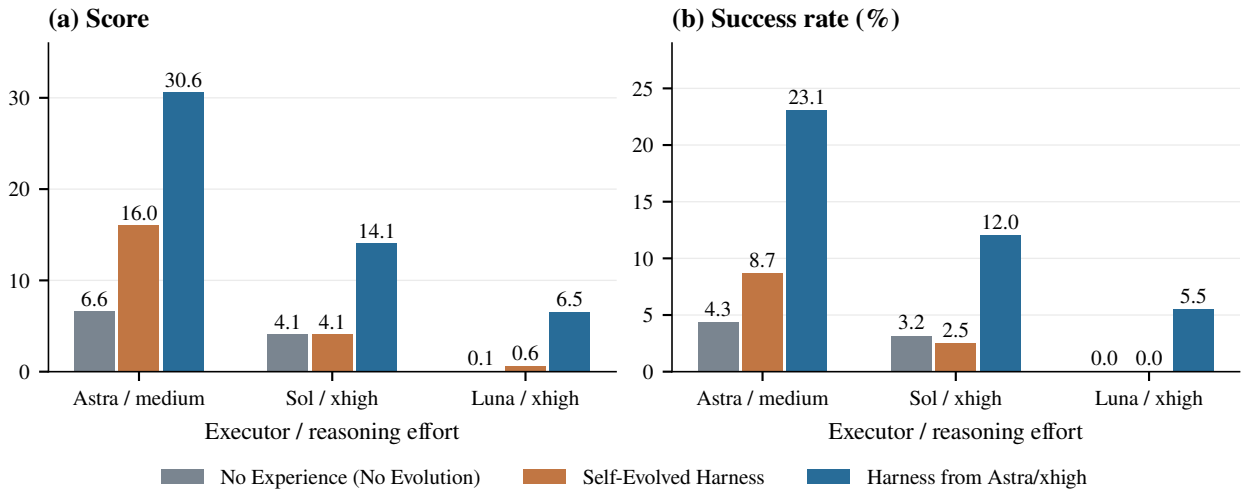


Figure 9 经验迁移至较弱的执行器。每个执行器在相同场景下，针对表 6 中使用的三种条件进行评估：无经验（无演化）、自演化引导器，以及来自 Astra/xhigh 的引导器。后者复用冻结的经验，无需额外探索。Astra/medium 在同一模型内比较源推理能力；Sol/xhigh 和 Luna/xhigh 测试跨模型复用。

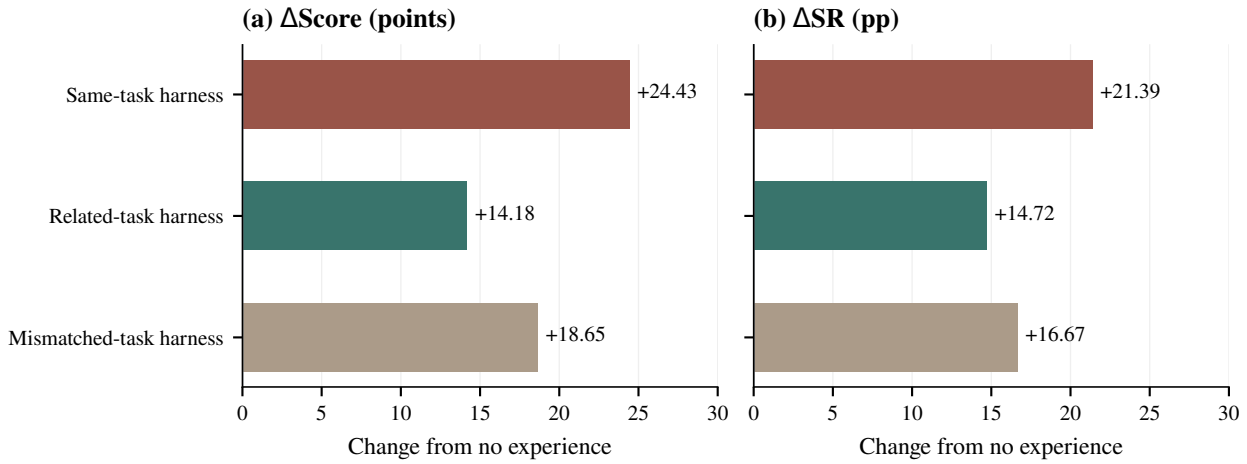


Figure 10 跨任务经验带来的性能提升。每个条形图表示在相同目标场景下，相较于无经验（No Experience）的改进，其中 Astra/xhigh 保持固定。Δ 得分使用分数，Δ SR 使用百分点；零值表示无经验的参考基准。在合并泛化变体后，十二个目标的得分权重相等。相关任务和不匹配任务的利用无需新的目标任务探索；同任务的利用作为参考。

`insert_tubes` 上，它们分别产生 0 和 +40 分。这两种映射使用相同的源池，且总文件量相等（附录 C）。这些反转结果表明，选择有用的经验需要考察保留的操作及其适用条件，而不仅仅是任务标签的语义相似度。

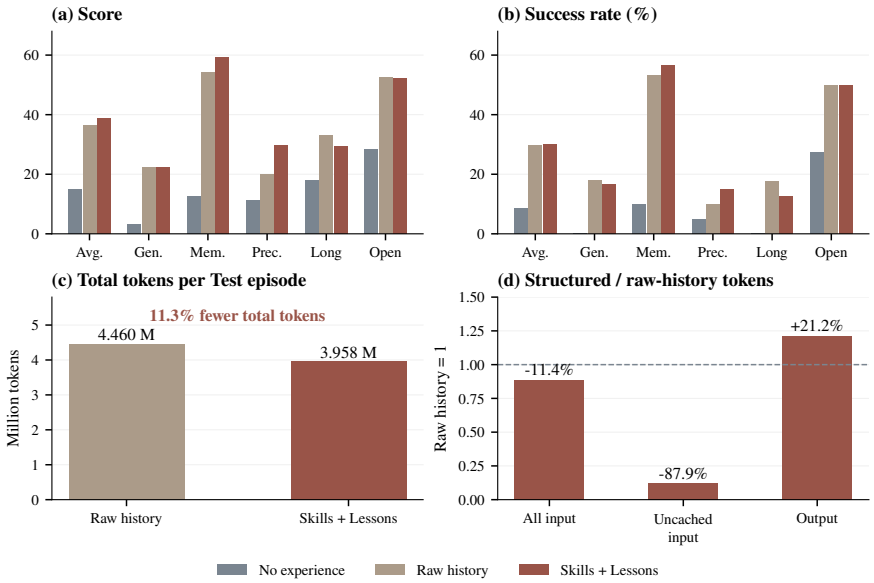


Figure 11 在使用更少 token 的情况下表现相当。执行器、源 Playground 和测试场景保持固定。(a–b) 原始历史记录与技能/经验在观测得分和成功率方面表现相似。(c) 技能/经验将总测试 token 数减少了 11.3%。(d) 减少部分来自输入使用量的降低，而输出则有所增加。token 数为每回合累计值，缓存读取包含在输入中。

4.5 经验表示与内容如何塑造重用

Research questions

(1) 紧凑的经验能否保留完整交互历史的优势？**(2)** 智能体在未完成任务的情况下能否获取有用的知识？**(3)** 什么使得保留的知识在后续执行中具有实用性？

为回答上述问题，我们对同一游乐场中的 Skills/Lessons 与 Raw History 进行受控对比，使用 Astra/x-high 在匹配的测试场景中进行评估。我们测量任务表现和总 token 使用量，然后检查未完成任务产生的结果，并将保存的程序和指导信息与后续执行关联起来。

技能/经验在使用更少 token 的情况下实现了相当的任务性能。原始历史记录提供了来自同一 Playground 的有序程序和公开反馈。其得分达到 36.41 / 成功率 29.78%，而技能/经验则达到 38.64 / 成功率 30.17%（图 11）。平均每回合的总测试使用量从 4.460M 下降至 3.958M token，减少了 11.3%。输入使用量从 4.445M 降至 3.939M，其中未缓存输入减少了 87.9%。技能/经验将探索组织为可重复使用的程序和接收方可直接查阅的明确指导。节省主要体现在输入方面，表明这种表示方式使先前交互的使用更加经济。附录 E 提供了配对估计值和 token 账目明细。

未完成的游乐场任务仍可产生有用的经验。在 `pick_from_conveyor_by_image` 上，Astra 在 100 次执行中均未记录到游乐场成功，但其冻结的夹具将测试得分从 0 提升至 40，成功率（SR）从 0 提升至 40%（表 7）。探索过程建立了有用的局部操作：第 74 次执行在 31 个原生动作内完成了一次追踪抓取和测试提升。保留的文件提供了动态目标捕获、接触保护运输，以及根据传送带移动更新目标的指导。即使整个任务仍未完成，这些发现仍可向前推进。经验积累在操作及其运行条件层面，使后续智能体能够基于部分进展继续构建。

有用的经验将可执行的修正与条件和检查相结合。在堆叠的迹中，测试执行 9 复用了具有 8 mm 平移约束的控制器；智能体检查了保留的负载，并在执行 15 时完成了堆叠（图 12）。在覆盖过程中，Astra 验

Table 7 **无 Playground 任务成功时的有用经验**。 Astra/xhigh 在图像条件约束的传送带抓取任务中；冻结的经验在新场景中进行评估。

Exploration	Observed progress and retained knowledge
Task outcome	No task success across 100 code executions.
Local progress	Execution 74: tracked grasp and 6 cm lift in 31 native actions.
Skills	Track a moving target through closure; monitor contact during transport.
Lessons	Anticipate conveyor motion; verify object retention after a short lift.
Test condition	Score / SR (%)
No experience	0 / 0
Skills/Lessons	40 / 40

证了对三个杯子的更高未覆盖抓取，并为空杯拾取和未覆盖操作分别记录了高度；Sol 应用了这些条件并在执行 17 时成功。这些例子将具体的修正、应用它的条件以及其效果的可观测检查联系起来。技能提供操作，而经验则指导其在当前场景中的选择和验证。附录 D.2 提供了保留的摘录和复用记录。

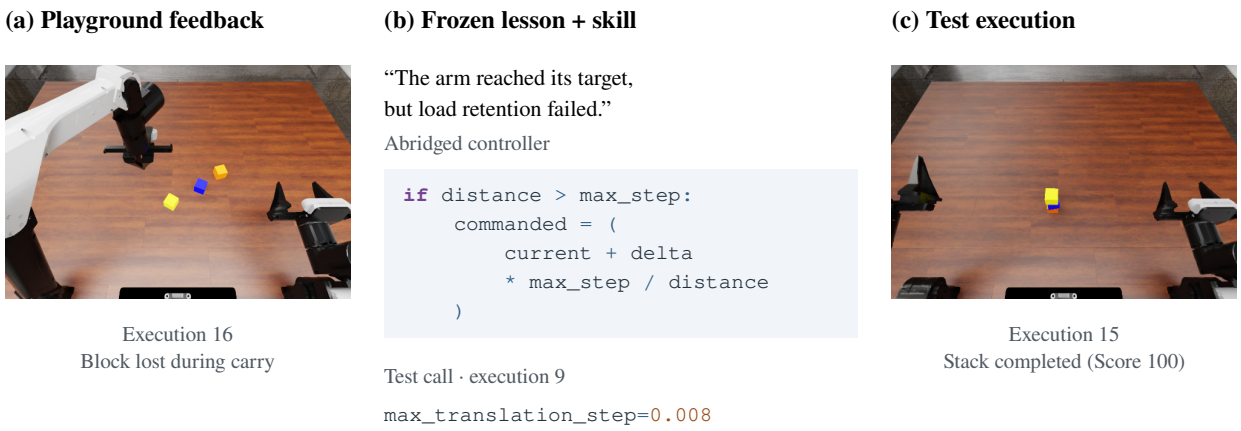


Figure 12 经过执行检查的修正。左图：游乐场执行 16 记录了负载掉落的情况。中图：冻结的课程区分了到达位姿与保持负载，控制器根据观测到的位姿限制运动。测试执行 9 以 8 mm 的约束调用它。右图：后续在执行 15 成功堆叠。控制器为简化的伪代码；课程片段和调用参数来自保留的成果和测试程序。

4.6 性能提升与重用经验的成本

Research questions

(1) 经验带来的收益伴随何种执行成本？**(2)** 对经验进行结构化能否降低重用成本？

为回答上述问题，我们在匹配的场景中对比了无经验、技能/教训以及原始历史与 Astra/xhigh 的表现。技能/教训和原始历史共享相同的 Playground 数据源。我们报告了每回合的模型开销和交互次数（包括失败情况），采用固定的 token 计费方案，且不包含 Playground 获取成本。

经验提升性能，但执行成本更高。得分从 14.77 提升至 38.64，成功率（SR）从 8.50% 提升至 30.17%，而每回合平均总测试成本从 $4.058 上升至 $5.732（+41.3%；表 8）。执行器在持续观察、提交代码并检查反馈的同时，会查阅保留的程序和指导。因此，经验提升了完成度，但仍需与环境保持持续交互。

Table 8 **固定执行器下的性能与执行开销**。 Astra/xhigh 在匹配的开发场景中的表现。成本和交互次数为每回合均值，包含失败情况。原始历史记录与技能/经验使用相同的 Playground 源。

Experience	Score	SR (%)	**Cost (USD)**	**Code executions**	**Native actions**
No Experience	14.77	8.50	4.058	22.3	723.0
Raw History	36.41	29.78	12.203	23.7	705.1
Skills/Lessons	38.64	30.17	5.732	29.0	722.6

结构化经验比原始历史更易于重用。使用相同的 Playground 源，Skills/Lessons 在每次 Test 回合的成本为 5.732 美元，而 Raw History 为 12.203 美元，降低了 53.0%，对应的 SR 分别为 30.17% 和 29.78%。Skills/Lessons 保留了可重用的程序及其应用条件，减少了在咨询先验探索时需处理的输入量。这种紧凑表示在保持相近观测性能的同时，显著降低了总执行成本。

5 讨论

代码使物理知识变得可检查和可修改。 代码揭示了执行物理策略所涉及的参数、检查和控制流程。这为反馈提供了明确的修订目标：在负载掉落之后，堆叠智能体仍保持有限运动，并在后续执行中检查物体是否被保留。参数化的过程可以在允许其目标随场景变化的同时，保持有用的操作。课程通过记录何时应用或重新考虑该策略来补充这种表示。原始历史记录实现了类似的观测成功率，而技能/课程将总测试 token 使用量减少了 11.3%。这表明，将交互组织为可访问的过程和指导是该表示的实际优势；仅靠代码并不能保证有效的物理假设。

演化改变了任务求解的起点。 在模型和机器人接口固定的情况下，演化会在新回合开始前改变可用的操作和决策条件。井字棋和排序案例展示了两种结果：可重复使用的抓取减少了重复的局部搜索，而保留的引导有助于智能体判断何时从观察转为操作。这些案例表明，演化将部分工作从发现操作转移到根据当前场景适应操作。然而，在零成功率任务和抓取回归中，这种转变不再起作用：接触条件和不完整流程仍需物理验证。因此，改进的有效单元是操作及其适用性证据的结合。

获取和应用经验是两种不同的能力。 Astra 生成的经验有助于 Sol 和 Luna 在自身探索收效甚微或毫无进展时取得进步。因此，智能体可以利用其难以自行获取的知识，实现一个生产者的探索成果支持多个执行者。跨任务结果进一步表明，复用效果取决于实体内容：所选的相关任务映射表现不如不匹配的映射。共享的动作检查与恢复指导可能适用于不同的标签任务。这促使我们根据接收者所需的操作和条件来选择经验，并结合当前观测进行适用性验证。

未来方向。 我们在 RoboDojo-Sim 上的评估为在更多基准、机器人平台和任务分布上测试测试工具演化提供了一个起点。长智能体调用延迟也促使执行效率的提升：未来工作可探索通过减少模型调用次数来复用已验证的流程，并将复杂的推理分配给不确定状态或恢复阶段。在评估成功率的同时衡量任务耗时，将有助于明确由此产生的响应速度-可靠性权衡。

6 结论

我们提出了具身演化（EmbodiedEvo）框架，使编码智能体能够将物理交互转化为可重用的技能与经验，而无需更新模型参数。在 RoboDojo-Sim 情景中，EmbodiedEvo 实现了 44.48 的平均得分和 37.82% 的成功率。成对评估显示，在不同输入设置下均表现出一致的性能提升。与原始交互历史相比，技能与经验在观测性能相当的情况下，测试执行成本降低了 53.0%。这些发现表明，通过演化优化可以改善任务求解的起始点，因为其保留了可执行的操作及其应用条件。精确的接触感知以及可靠的长时序执行仍是扩展该方法的重要方向。

References

[1] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024. URL https://arxiv.org/abs/2405.15793.

[2] Anthony Brohan, Noah Brown, Justice Carbajal, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023. URL https://arxiv.org/abs/2307.15818.

[3] Physical Intelligence, Kevin Black, Noah Brown, et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025. URL https://arxiv.org/abs/2504.16054.

[4] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. *arXiv preprint arXiv:2209.07753*, 2022. URL https://arxiv.org/abs/2209.07753.

[5] Letian Fu, Justin Yu, Karim El-Refai, et al. CaP-X: A framework for benchmarking and improving coding agents for robot manipulation. *arXiv preprint arXiv:2603.22435*, 2026. URL https://arxiv.org/abs/2603.22435.

[6] Zeyu Shen, Haoxiang You, Yilang Liu, et al. EmbodiedSWE: Coding agents for long horizon dexterous robotics. *arXiv preprint arXiv:2609.27308*, 2026. URL https://arxiv.org/abs/2609.27308.

[7] Wenbo Zhang, Kaixuan Wang, Yutao Ouyang, et al. An unexpected robot policy: Early evaluations of GPT-6 Astra on RoboDojo and beyond. *arXiv preprint arXiv:2609.24170*, 2026. URL https://arxiv.org/abs/2609.24170.

[8] Yifan Yang, Ziyang Gong, Weiquan Huang, Qihao Yang, Ziwei Zhou, Zisu Huang, Yan Li, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Yuqing Yang, Dongdong Chen, Xue Yang, and Chong Luo. SkillOpt: Executive Strategy for Self-Evolving Agent Skills. *arXiv preprint arXiv:2605.23904*, 2026. URL https://arxiv.org/abs/2605.23904.

[9] Seth Karten, Joel Zhang, Tersoo Upaa, Jr., Ruirong Feng, Wenzhe Li, Chengshuai Shi, Chi Jin, and Kiran Vodrahalli. Continual Harness: Online Adaptation for Self-Improving Foundation Agents. *arXiv preprint arXiv:2605.09998*, 2026. URL https://arxiv.org/abs/2605.09998.

[10] Ao Qu, Han Zheng, Zijian Zhou, Yihao Yan, Yihong Tang, Shao Yong Ong, Fenglu Hong, Kaichen Zhou, Chonghe Jiang, Minwei Kong, Jiacheng Zhu, Xuan Jiang, Sirui Li, Cathy Wu, Bryan Kian Hsiang Low, Jinhua Zhao, and Paul Pu Liang. CORAL: Towards Autonomous Multi-Agent Evolution for Open-Ended Discovery. *arXiv preprint arXiv:2604.01658*, 2026. URL https://arxiv.org/abs/2604.01658.

[11] Michał Zawalski, William Chen, Karl Pertsch, Oier Mees, Chelsea Finn, and Sergey Levine. Robotic control via embodied chain-of-thought reasoning. *arXiv preprint arXiv:2407.08693*, 2024. URL https://arxiv.org/abs/2407.08693.

[12] Physical Intelligence, Ali Amin, Raichelle Aniceto, et al. $\pi^*_{0.6}$: A VLA that learns from experience. *arXiv preprint arXiv:2511.14759*, 2025. URL https://arxiv.org/abs/2511.14759.

[13] Meng-Hao Guo, Zhe-Han Mo, Jia-Jun Wang, et al. Transferring the intelligence of VLMs to robotic control. *arXiv preprint arXiv:2609.22966*, 2026. URL https://arxiv.org/abs/2609.22966.

[14] Yuyao Liu, Jiayuan Mao, David Hsu, Leslie Pack Kaelbling, and Tomás Lozano-Pérez. RAPID: Robot agentic programming from demonstrations. *arXiv preprint arXiv:2609.30249*, 2026. URL https://arxiv.org/abs/2609.30249.

[15] So Kuroki and Yujin Tang. Learning and transferring closed-loop robot software. *arXiv preprint arXiv:2609.19906*, 2026. URL https://arxiv.org/abs/2609.19906.

[16] Hongcheng Gao, Jingjing Zhou, Zelin Zheng, Shijia Ge, Jay Zhu, Yazhe Wang, Jianshu Zeng, Xuan Shangguan, Di Wu, Lingyu He, Zhiqi Jia, Sihang Wu, and Xiao He. Self-evolving coding agents: From digital programs to physical-world intelligence. *arXiv preprint arXiv:2609.35432*, 2026. URL https://arxiv.org/abs/2609.35432.

[17] Chengsong Huang, Wenhao Yu, Xiaoyang Wang, Hongming Zhang, Zongxia Li, Ruosen Li, Jiaxin Huang, Haitao Mi, and Dong Yu. R-Zero: Self-Evolving Reasoning LLM from Zero Data. *arXiv preprint arXiv:2508.05004*, 2025. URL https://arxiv.org/abs/2508.05004.

[18] Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. SkillRL: Evolving Agents via Recursive Skill-Augmented Reinforcement Learning. *arXiv preprint arXiv:2602.08234*, 2026. URL https://arxiv.org/abs/2602.08234.

[19] Guibin Zhang, Haotian Ren, Chong Zhan, Zhenhong Zhou, Junhao Wang, He Zhu, Wangchunshu Zhou, and Shuicheng

Yan. MemEvolve: Meta-Evolution of Agent Memory Systems. *arXiv preprint arXiv:2512.18746*, 2025. URL https://arxiv.org/abs/2512.18746.

[20] Peidong Wang, Zhiming Ma, Ying Chang, et al. Self-evolving embodied agents via skill-harness evolution. *arXiv preprint arXiv:2608.11350*, 2026. URL https://arxiv.org/abs/2608.11350.

[21] Kailin Wang, Haoxiang Jie, Yaoyuan Yan, Zhiyou Heng, and Zhaosong Li. RegenHarness: A robot agent harness with evidence-gated recursive self-improvement. *arXiv preprint arXiv:2609.27612*, 2026. URL https://arxiv.org/abs/2609.27612.

[22] Jenny Zhang, Bingchen Zhao, Wannan Yang, Jakob Foerster, Jeff Clune, Minqi Jiang, Sam Devlin, and Tatiana Shavrina. Hyperagents. *arXiv preprint arXiv:2603.19461*, 2026. URL https://arxiv.org/abs/2603.19461.

[23] Chengsong Huang, Zifeng Wang, Rujun Han, Jun Yan, Yanfei Chen, Zoey CuiZhu, Ke Jiang, Peng Xia, Han Yu, Yufan Zhuang, Yifei Ming, Jiaqi Pan, Bhavana Dalvi Mishra, Jiaxin Huang, Burak Gokturk, Tomas Pfister, and Chen-Yu Lee. EnvHarness: Awakening Static Worlds for Agent Learning. *arXiv preprint arXiv:2608.19880*, 2026. URL https://arxiv.org/abs/2608.19880.

[24] Junyi Zhang, Jiaxin Ge, Hanjun Yoo, et al. Playful agentic robot learning. *arXiv preprint arXiv:2606.19419*, 2026. URL https://arxiv.org/abs/2606.19419.

[25] Sicheng Xie, Yitong Chen, Haidong Cao, Shunlin Lu, Zuxuan Wu, and Yu-Gang Jiang. Explore, execute, evolve: A skill acquisition and reuse loop for embodied agents. *arXiv preprint arXiv:2609.37810*, 2026. URL https://arxiv.org/abs/2609.37810.

[26] Runyu Lu, Yubo Wu, Ethan Kou, Letian Fu, Wenli Xiao, Ajay Mandlekar, Yinzhen Xu, Guanya Shi, Ken Goldberg, Ang Chen, Mosharaf Chowdhury, Yuke Zhu, Linxi Fan, and Guanzhi Wang. ASPIRE: Agentic /skills discovery for robotics. *arXiv preprint arXiv:2607.00272*, 2026. URL https://arxiv.org/abs/2607.00272.

[27] Shifeng Bao, Fanding Huang, Yihan Lin, et al. RoboHarn-Evo: Evolving hierarchical physical knowledge for self-improving robotic manipulation. *arXiv preprint arXiv:2609.37583*, 2026. URL https://arxiv.org/abs/2609.37583.

[28] Python Song, Zhixuan Liang, Kelsey Fu, Mengdi Wang, Junfeng Yang, and Shilong Liu. EmbodiedRSI: Active continual robot learning through hypothesis-guided co-evolution. *arXiv preprint arXiv:2610.10498*, 2026. URL https://arxiv.org/abs/2610.10498.

[29] HKU MMLab and KAI. PhysicalRSI 1.0: Recursive self-harness for scaling embodied skills. Project page, 2026. URL https://mmlab.hk/research/PhysicalRSI. Accessed October 12, 2026.

[30] RoboProbe. RoboProbe: An agentic robot manipulation codebase. GitHub repository, 2026. URL https://github.com/RoboProbe/RoboProbe.

[31] Tianxing Chen, Yue Chen, Zixuan Li, et al. RoboDojo: A unified sim-and-real benchmark for comprehensive evaluation of generalist robot manipulation policies. *arXiv preprint arXiv:2607.04434*, 2026. URL https://arxiv.org/abs/2607.04434.

[32] RoboProbe. GPT-6 Astra per-layout RoboDojo evaluation records. Public repository, revision `9ffacc54f372`, 2026. URL https://github.com/RoboProbe/RoboProbe/blob/9ffacc54f372beef6479d61711bf4945e31ae4ce/results/l3_inspect_eef_official_2100/astra_task_layouts.json.

[33] RoboProbe. RoboDojo task recipes. Repository snapshot, revision `523007520803`, 2026. URL https://github.com/RoboProbe/RoboProbe/tree/523007520803250860958 54e01977a8cf281b684/policy/RoboDojo_Agent_L3_Inspect/recipes. Fixed task descriptions, public scoring conditions, and upstream task notes used as experiment inputs.

A 游乐场和测试说明

Playground instruction

Goal
You are an embodied coding agent controlling a robot through code. You are currently in Playground Phase. Your goal is to explore the current robot task, develop a working solution, and extract reusable skills and lessons that may transfer to future scenes.
[...]
Organize your work as an iterative loop:
observe −> reason −> implement −> record reusable knowledge when it appears −> submit −> inspect −> revise
[...]
Whenever an observation reveals a reusable controller pattern, a repeated failure, a recovery strategy, or an important edge case, pause and write it to `skills/` or `lessons/` immediately while the evidence is fresh. Do not wait until the end of the session to record it.
Environment and Primitives
Read `primitives/primitives.md` before interacting with the environment. You have four native primitives:
— `get_instruction()`: read the current task and goal.
— `get_observation()`: read the latest observation, including camera images and robot state.
— `step(action, ...)`: execute a native controller action.
— `reset()`: restore the current scene and renew its official native action allowance. The total {{EXECUTION_BUDGET}} code−execution requests and elapsed time are never reset.
[...]
Evolution: skills and lessons
[...]
A skill should capture a general behavior or procedure, not merely rename a sequence of open−loop `step` calls. A useful skill should:
— be parameterized by relevant targets, tolerances, and control budgets;
— derive scene−dependent decisions from observations and explicit inputs;
— be reusable across similar tasks or scenes;
— specify its preconditions and limitations;
— include evidence from experiments that supports its behavior.
Lessons are textual knowledge about what worked, what failed, and how to recover. Record useful:
— failure patterns;
— diagnostic signals;
— recovery strategies;
— important edge cases;
— unresolved questions.
[...]
Playground rules
You may reset with `reset()` in code or `python scripts/env.py reset` and update `skills/` and `lessons/`.
Exploration uses the configured single scene. Success here is evidence about this scene; transfer to unseen scenes remains unverified.
Keep the task, primitives, and success conditions unchanged. Your transferable deliverables are the skills and lessons you develop.
You cannot use `reset()` in formal evaluation.
[...]

When done, out of slots, or unable to continue, review the current `skills/` and `lessons/` and run `finish`. If the connection breaks, query `status` before resubmitting.

Test instruction

Goal

You are an embodied coding agent controlling a robot through code. You are currently in Test Phase. Your goal is to complete the current robot task using the available primitives and the skills and lessons from Playground.

[...]

Organize your work as an iterative loop:

observe −> reason −> implement −> submit −> inspect −> revise

[...]

Environment and Primitives

Read `primitives/primitives.md` before interacting with the environment. You have three native primitives:

− `get_instruction()`: read the current task and goal.

− `get_observation()`: read the latest observation, including camera images and robot state.

− `step(action, ...)`: execute a native controller action.

[...]

Using skills and lessons

[...]

When using a skill:

− check its preconditions, limitations, and experimental evidence against the current observations;

− choose targets, tolerances, and control budgets appropriate to the current task;

− derive scene−dependent decisions from live observations and explicit inputs;

− inspect feedback after execution and decide the next stage from the resulting robot state.

Use lessons to recognize failure patterns, interpret diagnostic signals, and choose recovery strategies. Distinguish verified findings, scene−specific observations, and untested hypotheses; a result from Playground may not transfer to this scene.

[...]

Test rules

Work within a single continuous episode. You may observe, revise code, execute stages, and recover from failures using the current physical state. Both `reset()` in code and `python scripts/env.py reset` are forbidden in Test. Keep the task, primitives, success conditions, skills, and lessons unchanged. Complete the task using the frozen knowledge and the feedback from this scene.

Each Test scene starts with a fresh agent, workspace, and simulator session. Only frozen skills and lessons carry over from Playground; its executed program, temporary files, and observations do not. This scene's code, observations, and analysis do not carry over to other Test scenes.

[...]

Reset is forbidden in Test. All **{{EXECUTION_BUDGET}} code−execution requests** share the same **{{NATIVE_ACTION_LIMIT}} native action steps** for this scene; neither allowance can be restored.

[...]

When the task succeeds, you run out of execution slots, or you cannot continue, save episode−specific work in `submission/` and `scratch/` and run `finish`. The environment determines success. If the connection breaks, query `status` before resubmitting.

B 每项任务的结果

表 9 和 10 报告了完整的开发集和全集得分与成功率。开发集表格比较了 Astra 与 EmbodiedEvo 在有无维基百科信息情况下的匹配场景表现；全集表格报告了 EmbodiedEvo 在有维基百科信息的情况。泛化平均值对标准和随机变体同等对待，而平均值则对五个任务类别赋予相等权重。

Table 9 **每任务开发集结果**。每个单元格为得分 / SR (%)；所有四种条件均在匹配场景下使用 Astra/xhigh。阴影类别行报告均值；最后一行报告总体平均值。

Task	Task description		Task description + wiki	
	Astra	EmbodiedEvo	Astra	EmbodiedEvo
Generalization	**3.19 / 0.00**	**22.36 / 16.67**	**12.33 / 7.64**	**36.15 / 31.25**
arrange_largest_number	3.75 / 0.00	13.75 / 0.00	8.33 / 0.00	49.17 / 41.67
fold_clothes	11.67 / 0.00	13.33 / 0.00	33.33 / 16.67	70.00 / 66.67
hang_mugs	0.00 / 0.00	10.00 / 0.00	0.00 / 0.00	8.75 / 0.00
make_toast	0.00 / 0.00	0.00 / 0.00	0.00 / 0.00	0.00 / 0.00
pack_objects_into_box	6.67 / 0.00	22.50 / 0.00	6.67 / 0.00	22.50 / 0.00
pour_liquid_into_cup	0.00 / 0.00	0.00 / 0.00	0.00 / 0.00	41.67 / 41.67
push_T	0.00 / 0.00	83.33 / 83.33	0.00 / 0.00	83.33 / 83.33
sort_nesting_dolls_by_size	0.00 / 0.00	0.00 / 0.00	0.00 / 0.00	25.00 / 25.00
stack_blocks	10.00 / 0.00	100.00 / 100.00	50.42 / 41.67	100.00 / 100.00
stack_bowls	6.25 / 0.00	25.42 / 16.67	40.83 / 33.33	26.67 / 16.67
store_laptop_and_headphones	0.00 / 0.00	0.00 / 0.00	8.33 / 0.00	6.67 / 0.00
sweep_blocks	0.00 / 0.00	0.00 / 0.00	0.00 / 0.00	0.00 / 0.00
Memory	**12.67 / 10.00**	**59.33 / 56.67**	**39.67 / 36.67**	**70.33 / 66.67**
cover_blocks	6.00 / 0.00	86.00 / 80.00	60.00 / 60.00	100.00 / 100.00
imitate_sorting_sequence	10.00 / 0.00	90.00 / 80.00	38.00 / 20.00	62.00 / 40.00
match_and_pick_from_conveyor	60.00 / 60.00	100.00 / 100.00	100.00 / 100.00	100.00 / 100.00
press_by_number	0.00 / 0.00	0.00 / 0.00	20.00 / 20.00	100.00 / 100.00
swap_T	0.00 / 0.00	80.00 / 80.00	20.00 / 20.00	60.00 / 60.00
swap_blocks	0.00 / 0.00	0.00 / 0.00	0.00 / 0.00	0.00 / 0.00
Precision	**11.38 / 5.00**	**29.88 / 15.00**	**15.00 / 7.50**	**23.12 / 12.50**
build_tower	2.00 / 0.00	26.00 / 0.00	6.00 / 0.00	10.00 / 0.00
deposit_coin	8.00 / 0.00	12.00 / 0.00	8.00 / 0.00	36.00 / 20.00
fasten_screws	28.00 / 0.00	70.00 / 40.00	34.00 / 0.00	64.00 / 40.00
insert_key	9.00 / 0.00	15.00 / 0.00	12.00 / 0.00	15.00 / 0.00
insert_tubes	4.00 / 0.00	36.00 / 0.00	0.00 / 0.00	40.00 / 20.00
play_Xylophone	20.00 / 20.00	40.00 / 40.00	40.00 / 40.00	20.00 / 20.00
plug_in_charger	0.00 / 0.00	40.00 / 40.00	0.00 / 0.00	0.00 / 0.00
pour_balls_into_vase	20.00 / 20.00	0.00 / 0.00	20.00 / 20.00	0.00 / 0.00
Long-Horizon	**18.12 / 0.00**	**29.50 / 12.50**	**19.38 / 0.00**	**33.75 / 20.00**
classify_objects	22.00 / 0.00	40.00 / 0.00	30.00 / 0.00	47.00 / 20.00
fill_egg_holder	7.00 / 0.00	7.00 / 0.00	7.00 / 0.00	20.00 / 0.00
fill_pen_holder	15.00 / 0.00	5.00 / 0.00	8.00 / 0.00	2.00 / 0.00
make_kong	0.00 / 0.00	0.00 / 0.00	0.00 / 0.00	0.00 / 0.00
organize_table	15.00 / 0.00	40.00 / 0.00	40.00 / 0.00	30.00 / 0.00
play_stacking_toy	6.00 / 0.00	4.00 / 0.00	18.00 / 0.00	10.00 / 0.00
play_tic_tac_toe	55.00 / 0.00	100.00 / 100.00	32.00 / 0.00	100.00 / 100.00
put_bottles_into_dustbin	25.00 / 0.00	40.00 / 0.00	20.00 / 0.00	61.00 / 40.00
Open	**28.50 / 27.50**	**52.12 / 50.00**	**17.00 / 15.00**	**48.75 / 45.00**
align_blocks	60.00 / 60.00	80.00 / 80.00	20.00 / 20.00	100.00 / 100.00
classify_objects_by_language	24.00 / 20.00	48.00 / 40.00	16.00 / 0.00	46.00 / 20.00
general_pickup	80.00 / 80.00	60.00 / 60.00	40.00 / 40.00	80.00 / 80.00
pick_from_conveyor_by_image	0.00 / 0.00	40.00 / 40.00	20.00 / 20.00	20.00 / 20.00
pour_by_language	0.00 / 0.00	0.00 / 0.00	0.00 / 0.00	0.00 / 0.00
solve_equation	20.00 / 20.00	100.00 / 100.00	20.00 / 20.00	80.00 / 80.00
stack_blocks_by_language	44.00 / 40.00	84.00 / 80.00	20.00 / 20.00	64.00 / 60.00
store_tools_in_toolbox	0.00 / 0.00	5.00 / 0.00	0.00 / 0.00	0.00 / 0.00

Table 10 EmbodiedEvo (Astra/xhigh) **的每项任务完整结果**。任务描述 + wiki，来自表 3 中已完成的评估。阴影类别行报告均值；最后一行报告总体平均值。

Task / category	Score	SR (%)
Generalization	**34.12**	**29.67**
arrange_largest_number	58.20	46.00
fold_clothes	66.80	60.00
hang_mugs	5.40	0.00
make_toast	1.50	0.00
pack_objects_into_box	20.30	4.00
pour_liquid_into_cup	28.00	28.00
push_T	86.00	86.00
sort_nesting_dolls_by_size	8.00	8.00
stack_blocks	71.60	68.00
stack_bowls	54.50	50.00
store_laptop_and_headphones	3.20	0.00
sweep_blocks	6.00	6.00
Memory	**71.90**	**69.67**
cover_blocks	100.00	100.00
imitate_sorting_sequence	71.40	58.00
match_and_pick_from_conveyor	76.00	76.00
press_by_number	98.00	98.00
swap_T	86.00	86.00
swap_blocks	0.00	0.00
Precision	**23.65**	**13.25**
build_tower	8.60	0.00
deposit_coin	37.20	22.00
fasten_screws	60.00	38.00
insert_key	15.00	0.00
insert_tubes	36.40	14.00
play_Xylophone	16.00	16.00
plug_in_charger	4.00	4.00
pour_balls_into_vase	12.00	12.00
Long-Horizon	**37.20**	**23.50**
classify_objects	44.30	22.00
fill_egg_holder	13.80	0.00
fill_pen_holder	2.00	0.00
make_kong	0.00	0.00
organize_table	41.50	2.00
play_stacking_toy	18.80	0.00
play_tic_tac_toe	100.00	100.00
put_bottles_into_dustbin	77.20	64.00
Open	**55.52**	**53.00**
align_blocks	98.00	98.00
classify_objects_by_language	63.40	46.00
general_pickup	86.00	86.00
pick_from_conveyor_by_image	38.00	38.00
pour_by_language	0.40	0.00
solve_equation	76.00	76.00
stack_blocks_by_language	82.40	80.00
store_tools_in_toolbox	0.00	0.00
Average	**44.48**	**37.82**

C 跨任务迁移

表 11 给出了第 4.4 节中十二个目标的源分配和结果。相关分配将任务与相似操作配对；非匹配分配则打乱了相同的源池。两个映射在执行前已固定，并暴露了总计 388,507 字节的 146 个文件，每个目标包仅计算一次。仅传输只读技能/课程；指令、目标和场景保持为目标的原样。两种传输条件均不执行目标-任务探索。

所有条件均在相同的目标场景上使用 Astra/xhigh。泛化变体在每个目标内获得相等权重，随后十二个目标也获得相等权重。该表格保留绝对结果，以便重建图 10 中相对于无经验的增益；这些子集均值与全基准平均值分离。

Table 11 **跨任务源与目标结果**。每个结果均为得分 / SR (%)。相关和不匹配的源在这些迁移运行前已固定；B 和 D 重复使用相同的目标场景结果。

Target	Related source	Mismatched source	B		D		Related		Mismatched	
			Score	SR (%)	Score	SR (%)	Score	SR (%)	Score	SR (%)
stack bowls	stack blocks	swap blocks	6.25	0.00	25.42	16.67	25.42	16.67	15.00	0.00
stack blocks	stack bowls	swap T	10.00	0.00	100.00	100.00	100.00	100.00	83.33	83.33
insert tubes	plug in charger	classify objects	4.00	0.00	36.00	0.00	60.00	40.00	4.00	0.00
plug in charger	insert tubes	classify objects by language	0.00	0.00	40.00	40.00	0.00	0.00	0.00	0.00
fill pen holder	fill egg holder	pour liquid into cup	15.00	0.00	5.00	0.00	2.00	0.00	17.00	0.00
fill egg holder	fill pen holder	pour by language	7.00	0.00	7.00	0.00	8.00	0.00	14.00	0.00
swap blocks	swap T	stack bowls	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
swap T	swap blocks	stack blocks	0.00	0.00	80.00	80.00	0.00	0.00	100.00	100.00
classify objects	classify objects by language	insert tubes	22.00	0.00	40.00	0.00	39.00	20.00	30.00	0.00
classify objects by language	classify objects	plug in charger	24.00	20.00	48.00	40.00	24.00	20.00	32.00	20.00
pour liquid into cup	pour by language	fill pen holder	0.00	0.00	0.00	0.00	0.00	0.00	16.67	16.67
pour by language	pour liquid into cup	fill egg holder	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
Fixed-target mean			7.35	1.67	31.78	23.06	21.53	16.39	26.00	18.33

备注： B：无经验；D：同任务测试。在每个泛化任务内，标准/随机变体平均处理，随后十二个目标任务权重相等。这些子集均值与完整基准平均值分离。所有执行器和经验生成器均为 Astra/xhigh。两种迁移条件下均未运行新的 Playground。

D 支持性案例证据

D.1 能力变更

表 12 总结了第 4.3 节背后的执行记录。所有案例均使用 Astra/xhigh；成对比较使用相同场景。证据包保留了原始程序、反馈、文件读取和相机帧。这些选取的案例展示了机制，而非故障频率或孤立因果效应。

Table 12 能力分析的执行证据。动作计数指原生模拟器动作；布局 ID 识别匹配的场景。

Task / layout	Key execution evidence
Tic-tac-toe / 27	Astra first retains a ring at action 367 and ends with Score 50 at the 1,100-action limit. EmbodiedEvo includes `rim_pick.py`, retains a ring at action 38, and succeeds at action 919. First rings and arms differ.
Imitation sorting / 4	Astra moves before the demonstration finishes and terminates at action 256 with Score 0; feedback does not identify the cause. EmbodiedEvo observes the new order through action 455 and succeeds at action 1,585.
Insert key / 44	EmbodiedEvo's receiver closes empty in execution 14 and remains unsecured through execution 21. Donor release reaches the 300-action limit with Score 15. Failure occurs during handover, before insertion.

D.2 经验的复用及其局限性

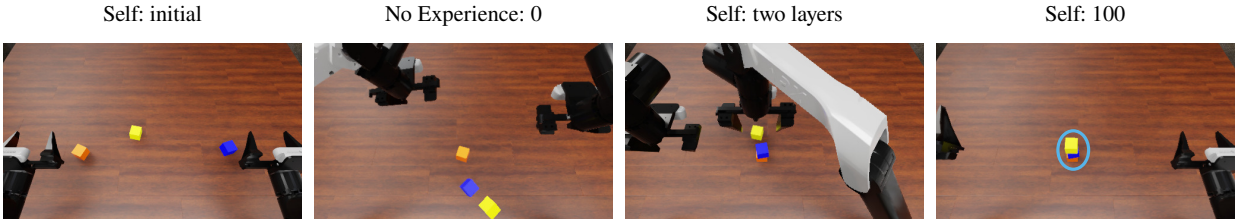
图 13 比较了成功的复用、跨模型迁移和回归。图 14 提供了完整的保留抓取函数以及在附录 F 中成功堆叠迹中使用的两个教学示例。

堆叠：**一种带有核对的校正**。 冻结的课程记录显示："手臂达到了目标，但负载保持失败。"测试执行 2 包含 `ee_move.py` 和 `pick_from_above.py` 的完整代码。执行 9 在视觉保持检查后使用了 8 mm 的平移约束；执行 15 完成了堆叠。这确立了实际的重复使用，而未将约束的因果作用孤立出来。

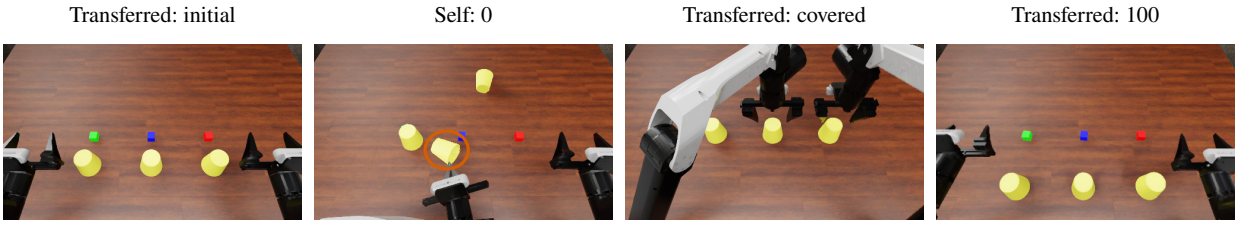
涵盖：**传递条件**。 索尔自身的装置描述了一个"仅限拾取的实验性技能，而非经过验证的抓取或放置控制器"；在运输过程中，其测试丢失了杯子。而阿斯特拉的课程则区分了空杯拾取与更高层次的揭盖抓握，并在三个杯子上验证了该能力。索尔在执行 2、11 和 13 中使用了这些分离的高度，并在执行 17 中成功完成。

拾取：**一个传输边界**。 Astra 的保存回缩提升在游乐场的剪刀上成功。在大象上，测试执行 7 报告 `pose_reached`，但后续观察显示物体已丢失。因此，源场景的成功使得新几何体上的保留问题仍未解决；迹线无法将保存的建议确定为失败的原因。

(a) Language-conditioned stacking · Astra/xhigh · layout 37



(b) Cover and uncover by color · Sol/xhigh · layout 38



(c) Elephant pickup · Astra/xhigh · layout 15

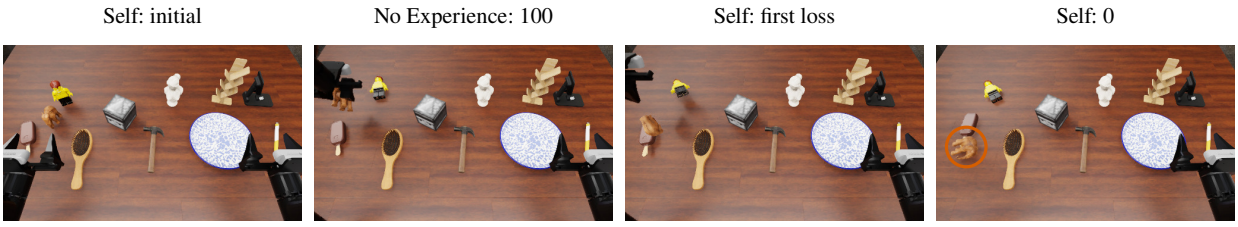


Figure 13 成功、迁移与回归在匹配场景中的表现。 (a) 语言堆叠，Astra/xhigh，布局 37：自持系统完成堆叠。(b) 覆盖/揭开，Sol/xhigh，布局 38：Astra 的持握系统成功，而 Sol 自身的持握系统失败。(c) 大象拾取，Astra/xhigh，布局 15：Astra 在无经验情况下成功，但其自持系统导致物体掉落。帧来自独立运行的原始观测，裁剪部分相同。标签显示官方得分和执行索引。

(a) Skill: a guarded, budgeted grasp function

```
def pick_from_above(arm, grasp_pose, lift_z, action_budget, grip_dwell=12, descend_step=0.010, lift_step=0.008,
  alignment_tolerance=0.003):
    """Pick from an already aligned hover; return for visual retention verification."""
    obs = get_observation()
    current = obs['state'][arm + '_ee_pose']
    lateral_error = sum((current[j] - grasp_pose[j]) ** 2 for j in range(2)) ** 0.5
    if lateral_error > alignment_tolerance or current[2] < grasp_pose[2]:
        return obs, {'error': 'requires_aligned_hover', 'steps': 0}
    used = 0
    stages = [('descend', list(grasp_pose), 1.0, 3, descend_step),
              ('close', list(grasp_pose), 0.0, grip_dwell, descend_step)]
    raised = list(grasp_pose)
    raised[2] = lift_z
    stages.append(('lift', raised, 0.0, 3, lift_step))
    for name, target, grip, dwell, speed in stages:
        remaining = action_budget - used
        if remaining < dwell:
            return obs, {'error': 'budget_exhausted', 'stage': name, 'steps': used}
        obs, result = move_ee(arm, target, grip, max_steps=remaining, min_steps=dwell, max_translation_step=speed)
        used += result['steps']
        if not result.get('reached', False):
            return obs, {'stage': name, 'steps': used, 'motion': result}
    return obs, {'stage': 'lifted_needs_visual_check', 'steps': used, 'gripper': obs['state'][arm + '_ee_joint_state']}
```

(b) 步骤：验证保持，然后缓慢搬运

尽管机械臂已到达目标位置，但搬运过程中仍掉落了方块。保存的规则是：对每次平移进行约束，并在每个搬运阶段后检查负载情况。测试执行 9 在智能体检查被举起的蓝色方块后，采用 8 mm 的约束。

```
obs, result = move_ee('right', base_xy+[1.045]+down, 0.0,
  max_steps=46, max_translation_step=0.008)
print(result)
```

(c) 课程：将动作保留至最终检查

保存的规则要求在放置后执行返回动作，并在终止时立即停止。稳定的测试堆栈在执行 14 时仍为 success=false。执行 15 触发 success，经过 10 次返回动作，还剩余五个动作。

```
for i in range(15):
    obs, reward, terminated, truncated, info = step(home)
    if terminated or truncated:
        print({'steps':i+1,'reward':reward,'terminated':
  terminated,'truncated':truncated,'success':info.get('success',
  False)})
        break
```

Figure 14 实际复用中的保留技能与两个教训。 (a) 完整冻结的 `pick_from_above` 函数将测量位姿 `move_ee` 控制器组合为下降、闭合和提升。它检查对齐状态和动作预算，然后返回以进行视觉保留验证。同一源代码在测试执行 2 中原样包含，并在执行 4、8 和 12 中分别调用以处理橙色、蓝色和黄色。(b–c) 教训描述对冻结的 `downward_approach.md` 进行了改写；突出显示的测试片段为原文。完整回合见附录 F。

E 原始历史对比与成本核算

匹配比较。 原始历史（Raw History）通过只读文件提供与沙盒环境相同的有序程序和公开反馈，包括原始观测结果和失败尝试，但不包含技能/课程以及私有得分，且不会由其他模型进行汇总。两种表示均在匹配的测试场景中使用 Astra/xhigh 进行评估，且不进行重置。相较于原始历史，技能/课程在得分上高出 2.23 分（95% 置信区间：−2.60 –7.25），在成功率（SR）上高出 0.39 分（−5.11 –5.94）。置信区间基于每类任务内 10,000 次配对任务自举抽样（种子号 20261006）计算得出。观察到的性能相似并不构成正式等价性结论。

代币与费用。 图 11 和表 8 展示了完整测试回合中终端 API 使用计数器的平均值，包含失败情况和重复上下文；排除了 Playground 获取。总 token 数等于输入加输出，其中缓存读取计入输入，推理部分计入输出。某一本地原始迹线中缺失的最终事件通过其终端报告进行恢复，未采用插补。缓存读取占原始迹线输入的 82.6%，占技能/经验输入的 97.6%。固定 Astra 费率分别为每百万普通输入/缓存读取/缓存写入/输出 token 10 美元/1 美元/12.5 美元/50 美元。这些名义费率解释了成本节省（53.0%）大于总 token 节省（11.3%）的原因；相关数值为估算值，非正式账单。

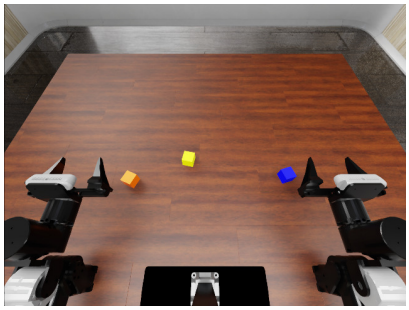
F 完整成功智能体迹

此迹线涵盖了 Astra/xhigh 自动测试运行中 `stack_blocks_by_language` 的每一次被接受的执行，公开布局 37。该回合共发起 15 次执行请求，使用了 395 / 400 个原生动作，最终官方得分 100 且 success=true。以下每个移动阶段均配有一张智能体实际在显示动作之前打开的摄像头图像，以及其提交的决策注释、高亮代码和后续反馈。图像保留其原始视场。执行 1 获取指令与初始图像读取；执行 2–15 在同一仿真会话中推进，未重置。

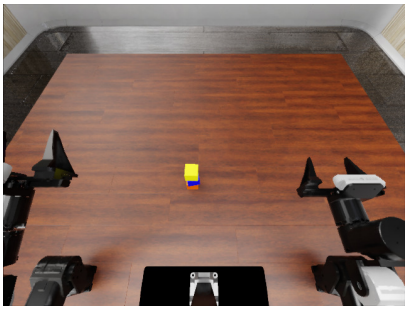
智能体所阅读的内容。 智能体读取固定的测试委托、原生基本操作手册、技能索引、`ee_move` 和 `pick_from_above` 的源文件及使用说明，以及 `lessons/downward_approach.md`。它还读取初始状态元数据并打开头戴摄像头图像。该会话在第 18、25 和 32 行记录了这些访问操作。冻结的源代码被添加到执行 2 中；后续调用复用持久化的 Python 命名空间。第二个辅助函数如图 14 所示。执行 7 添加了一个回合局部的 `move_both` 辅助函数，用于将空置的手臂一同重新定位。

> **`get_instruction()` 返回的任务**：“从下到上按橙色、蓝色和黄色的顺序堆叠块，然后重置机械臂。”

此处“重置机械臂”指的是命令保存的起始关节，正如执行 15 中通过 `step(home)` 所做的那样。测试回合从未调用环境的 `reset()` 原语。



初始观察：在对照组之前打开



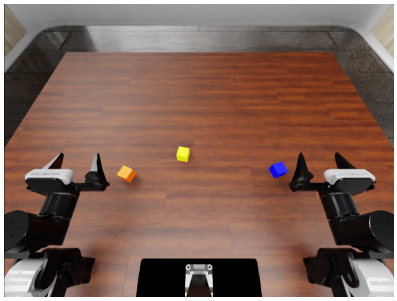
终端观察：在执行 15 后记录

Figure 15 完整堆叠回合的端点。初始头部视角由智能体打开；终端图像由环境记录。执行第 15 步后，智能体接收到成功反馈并结束，不再进行进一步的图像打开调用。所需的最终顺序为从下到上：橙色–蓝色–黄色。

阅读序列。 决策文本复制自智能体提交的代码注释；执行 1 对其指令调用有一个编辑性描述。这些注释是针对某次动作所记录的理由，并非对图像所证明内容的独立标注。反馈文本汇总了后续的标准输出（stdout）及公开执行计数器。代码面板展示了完整的、按阶段划分的动作代码，但执行 2 中重复导入的辅助函数以及执行 7 中定义的局部辅助函数除外。源代码包保留了所有提交的程序、完整标准输出、冻结的辅助函数与课程内容，以及位于 `agent-trace/trace.json` 中的图像访问索引。

执行顺序

执行 1：阅读任务 +0 actions; 0/400 used



头部摄像头

观察 0，指令检索过程中

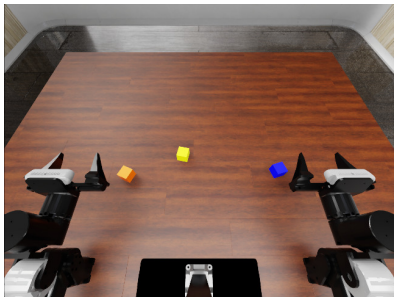
已记录的决策。在发出机器人动作之前，获取当前的任务指令。

```
print(get_instruction())
```

反馈。指令中指定的颜色顺序为橙色、蓝色，然后是黄色；未使用原生动作。

执行 2：加载技能并接近橙色

+22 actions; 22/400 used



头部摄像头

观察 0，执行前 2

记录的决策。橙色必须作为底座，蓝色和黄色在其上方，然后返回家中。保留家关节，以抬高的姿态接近橙色，以安全地校准其位置。

```
obs = get_observation()
home = {k:list(v) for k,v in obs['state'].items() if k.endswith('joint_state')}
down = [0.5,−0.5,0.5,0.5]
obs, result = move_ee('left', [−0.20,−0.10,1.10]+down, 1.0, max_steps=45)
print(result)
print(obs['state'])
```

反馈。目标被抬高，在 22 个动作后达到；初始的关节字典已保存。

执行 3：正确对齐橙色

+7 actions; 29/400 used



左腕相机

观察 2，执行 3 之前

已记录的决策。在此高度，橙色物体位于手腕校准抓取对齐位置的左侧且向前方。在下降之前，先将抬起的左侧夹爪移向它。

```
obs, result = move_ee('left', [−0.225,−0.17,1.10]+down, 1.0, max_steps=12)
print(result)
```

反馈。在 7 个动作内达到了提升的对齐目标。

执行 4：抓取并举起橙色物体

+55 actions; 84/400 used



左腕相机

观察 3，执行 4 之前

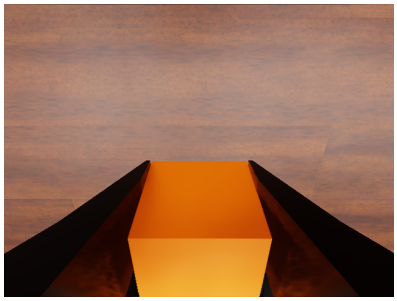
记录的决策。抬腕视图将橙色定位在 (-0.231, -0.188) 附近。精确对齐后，使用已验证的低抓取和缓慢垂直提升，再检查保持情况。

```
obs, result = move_ee('left', [−0.231,−0.188,1.10]+down, 1.0, max_steps=6)
print('alignment', result)
if result.get('reached', False):
    obs, result = pick_from_above('left', [−0.231,−0.188,0.925]+down, 1.065,
  action_budget=52)
    print('pick', result)
print(obs['state']['left_ee_pose'])
```

反馈。对齐使用 3 个动作，抓取使用 52 个动作。该技能返回 lifted_needs_visual_check。

执行 5：携带保留的橙色块

+28 actions; 112/400 used



左腕相机

观察 4，执行 5 之前

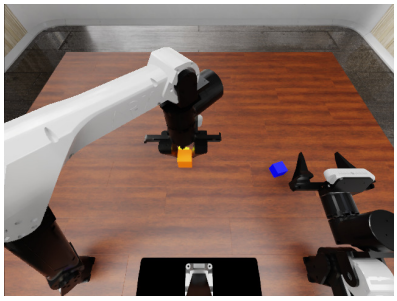
已记录的决定。在抬起时，橙色物体在左手指之间仍明显可见。将其缓慢移向中央基底位置，同时保持高于桌面的间隙。

```
base_xy = [−0.035,−0.188]
obs, result = move_ee('left', base_xy+[1.045]+down, 0.0, max_steps=34,
  max_translation_step=0.008)
print(result)
print(obs['state']['left_ee_pose'])
```

反馈。 8 毫米的平移范围在 28 个动作内达到中心基底位置。

执行 6：放置底座并后退

+35 actions; 147/400 used



头部摄像头

观察 5，执行 6 之前

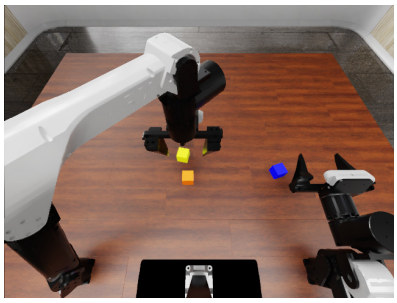
记录的决策。橙色保持在中心位置，下方有清晰的平台空间。将其降低至平台高度，停留后垂直撤回。

```
obs, result = move_ee('left', base_xy+[0.928]+down, 0.0, max_steps=23,
  max_translation_step=0.008)
print('lower', result)
if result.get('reached', False):
    obs, result = move_ee('left', base_xy+[0.928]+down, 1.0, max_steps=8,
  min_steps=8)
    print('release', result)
    if not result.get('terminated', False) and not result.get('truncated', False):
        obs, result = move_ee('left', base_xy+[1.065]+down, 1.0, max_steps=15,
  max_translation_step=0.018)
        print('retreat', result)
```

反馈。下降、释放和撤退分别需要 17、8 和 10 个动作。

执行 7：清理左臂并靠近蓝色物体

+12 actions; 159/400 used



头部摄像头

观察 6，执行前 7

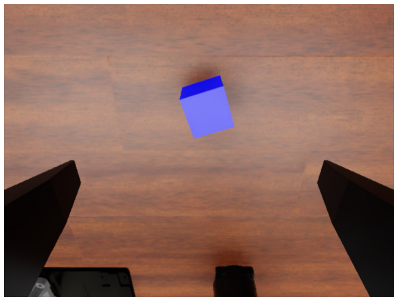
记录的决策。橙色在桌子上保持稳定，蓝色在堆叠中紧邻其下。将空闲的左臂移开，同时右臂以安全距离接近蓝色物体。

```
obs, result = move_both([−0.24,−0.24,1.065]+down, [0.25,−0.16,1.065]+down,
  max_steps=22)
print(result)
print(obs['state']['right_ee_pose'])
```

反馈。一个局部定义的 move_both 辅助函数在 12 个动作内到达两个目标。

执行 8：抓取并抬起蓝色物体

+48 actions; 207/400 used



右腕摄像头

观察 7，在执行 8 之前

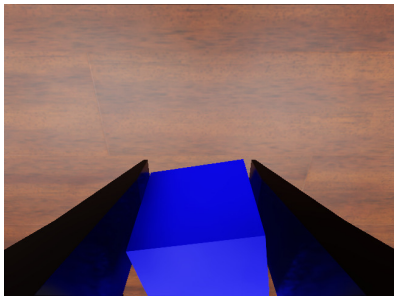
记录的决策。蓝色略偏右且向前于校准的手腕对齐位置。纠正抬高的姿势，于桌面高度抓握，并缓慢抬起以验证保持情况。

```
blue_xy = [0.2575,−0.166]
obs, result = move_ee('right', blue_xy+[1.065]+down, 1.0, max_steps=5)
print('align',result)
if result.get('reached',False):
    obs, result = pick_from_above('right', blue_xy+[0.925]+down, 1.045,
  action_budget=48)
    print('pick',result)
```

反馈。对齐需要 3 个动作，继承的抓取需要 45 个；仍需进行视觉验证。

执行 9：将蓝色块移至橙色块上方

+41 actions; 248/400 used



右腕摄像头

观察 8，执行 9 之前

已记录的决策。提升后蓝色仍明显保留，橙色在中心基底处稳定。以测试载荷速度将蓝色运送到基底对齐位置，进行第二层放置。

```
obs, result = move_ee('right', base_xy+[1.045]+down, 0.0, max_steps=46,
  max_translation_step=0.008)
print(result)
print(obs['state']['right_ee_pose'])
```

反馈。在 8 毫米平移约束下，装载的携带物在 41 次动作内到达目标。

执行 10：放置第二层

+34 actions; 282/400 used



头部摄像头

观察 9，执行前 10

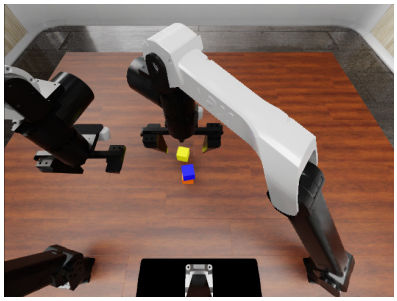
已记录的决策。蓝色保持在橙色正上方，手腕对齐与校准参考一致。下降至第二层释放高度，打开并垂直退后以检查堆叠。

```
obs, result = move_ee('right', base_xy+[0.968]+down, 0.0, max_steps=23,
  max_translation_step=0.005)
print('lower',result)
if result.get('reached',False):
    obs, result = move_ee('right', base_xy+[0.968]+down, 1.0, max_steps=8,
  min_steps=8)
    print('release',result)
    if not result.get('terminated',False) and not result.get('truncated',False):
        obs, result = move_ee('right', base_xy+[1.075]+down, 1.0, max_steps=10,
  max_translation_step=0.018)
        print('retreat',result)
```

反馈。下降、释放和后退分别需要 18、8 和 8 个动作。下一帧显示蓝色在橙色上。

执行 11：清除右臂并靠近黄色物体

+15 actions; 297/400 used



头部摄像头

观察 10，执行 11 之前

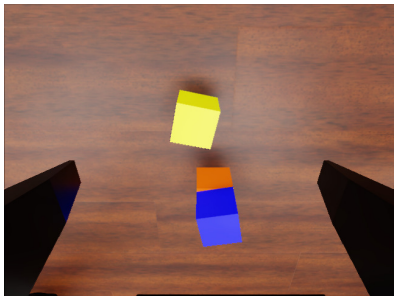
记录的决策。蓝色在橙色上保持稳定，留下黄色作为最终层。移动空闲的右臂至安全位置，同时将左臂对准黄色上方进行视觉校准。

```
obs, result = move_both([−0.05,−0.09,1.065]+down, [0.24,−0.24,1.075]+down,
  max_steps=20)
print(result)
```

反馈。两个提升的目标在 15 个动作内达成；还剩余 103 个原始动作。

执行 12：抓取并举起黄色物体

+50 actions; 347/400 used



左腕相机

观察 11，执行前 12

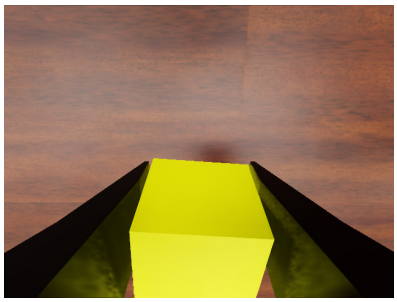
已记录的决策。黄色块略微位于桌高腕部对齐参考线的左侧和前方。将其中心调整正确，低位抓取，并垂直抬起，确保高于双块堆叠。

```
yellow_xy = [−0.0535,−0.110]
obs, result = move_ee('left', yellow_xy+[1.065]+down, 1.0, max_steps=5)
print('align',result)
if result.get('reached',False):
    obs, result = pick_from_above('left', yellow_xy+[0.925]+down, 1.045,
  action_budget=48)
    print('pick',result)
```

反馈。对齐需要 3 个动作，抓取需要 47 个；还剩下 53 个原生动作。

执行 13：将黄色方块移至可见的最上方方块之上

+12 actions; 359/400 used



左腕相机

观察 12，执行 13 之前

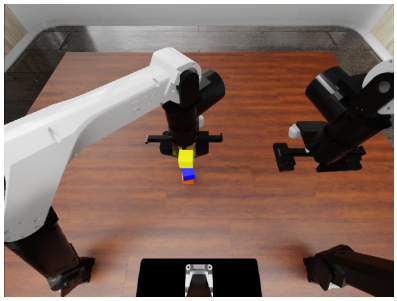
记录的决策。黄色被明显保留，橙色-蓝色堆叠保持直立。将黄色移至测量堆叠中心，使用测试的慢速平移极限。

```
top_xy = [−0.033,−0.188]
obs, result = move_ee('left', top_xy+[1.045]+down, 0.0, max_steps=15,
  max_translation_step=0.008)
print(result)
```

反馈。已在 12 个动作内达到修正后的堆栈中心目标；还剩 41 个动作。

执行 14：放置黄色并保留总回报储备

+26 actions; 385/400 used



头部摄像头

观察 13，执行前 14

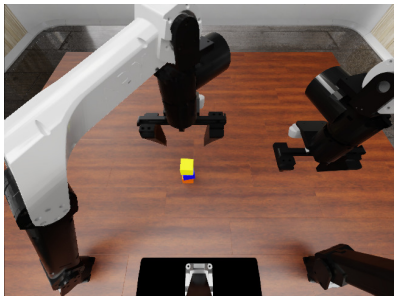
记录下的决策。黄色块置于实际蓝色顶层块之上，橙色块支撑着堆叠结构。缓慢放置最后一层，释放后，在返回原点检查前撤离。

```
obs, result = move_ee('left', top_xy+[1.008]+down, 0.0, max_steps=13,
  max_translation_step=0.004)
print('lower',result)
if result.get('reached',False):
   obs, result = move_ee('left', top_xy+[1.008]+down, 1.0, max_steps=8, min_steps
  =8)
   print('release',result)
   if not result.get('terminated',False) and not result.get('truncated',False):
      obs, result = move_ee('left', top_xy+[1.11]+down, 1.0, max_steps=9,
  max_translation_step=0.018)
      print('retreat',result)
```

反馈。 4 毫米下降、8 动作释放和后退共使用 26 个动作。成功仍为假；还剩余 15 个动作。

执行 15：返回家中并停在官方成功状态

+10 actions; 395/400 used



头部摄像头

观察 14，执行前 15

记录的决策。相机显示释放和垂直提起后，堆叠稳定地呈现橙色-蓝色-黄色。尽快将双臂恢复到保存的起始关节位置，并在官方检查结束时停止。

```
for i in range(15):
   obs, reward, terminated, truncated, info = step(home)
   if terminated or truncated:
      print({'steps':i+1,'reward':reward,'terminated':terminated,'truncated':truncated,'
  success':info.get('success',False)})
      break
print(obs['state'])
```

反馈。家庭命令在执行 10 个动作后触发 success=true 且 terminated=true。还剩下五个动作未使用。